

SingProbe Technical Report

Sing Team
AI Security Lab, Ant Group

<https://github.com/inclusionAI/SingProbe>
<https://huggingface.co/collections/inclusionAI/singprobe>

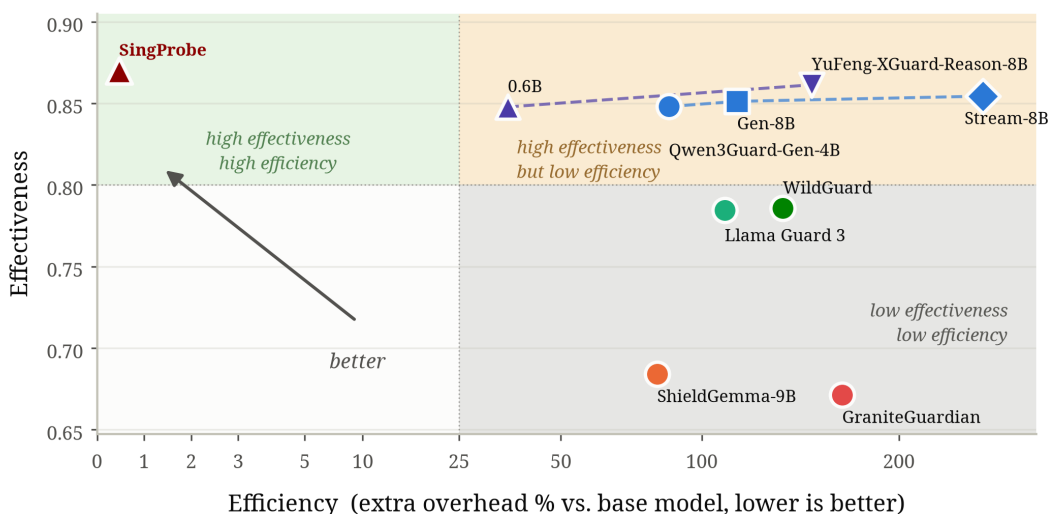


Figure 1: Effectiveness–efficiency trade-off of guardrail models. Efficiency (x) is the extra inference latency of a guard over the base-model-only serving baseline. We use Ling-3.0-flash as the base model. A non-streaming guard is invoked once after the complete response has been generated, whereas a streaming guard is invoked at every generation step. Effectiveness (y-axis) is measured as the average of the guard’s Query F1 and Response F1 across the evaluated benchmarks.

Abstract

Runtime guardrails are essential for reliable large language model (LLM) deployment, yet existing approaches typically rely on independent, external models that introduce additional inference cost, delayed safety signals, and a capacity mismatch with increasingly capable base models. To address these issues, we introduce SingProbe, a lightweight intrinsic runtime guard that directly reuses hidden states produced during LLM inference and operates alongside autoregressive decoding. Within a unified framework, SingProbe continuously predicts query intent, response safety, and hallucination risk at the token level with negligible additional guardrail inference overhead, offering a "free-lunch" solution. We further introduce SingStreamBench, a benchmark designed to assess whether streaming guardrails remain inactive on benign prefixes while promptly detecting emerging unsafe content. Extensive experiments show that SingProbe achieves competitive or superior performance compared with substantially larger standalone guardrails and specialized hallucination detectors, with only $\approx 3\text{--}5\text{M}$ parameters and $< 0.5\%$ extra overhead. Beyond passive detection, we also show that SingProbe scores can anticipate future generation risk and guide constrained safe decoding. We further extend this paradigm to medical generation through SingProbe-Med, which selectively activates risk-directed decoding interventions only when

clinically relevant risks emerge. Together, these results demonstrate that internal model representations provide an effective and efficient interface for generation-time monitoring and control.

1 Introduction

As LLMs are increasingly deployed in diverse real-world applications, unsafe, harmful, or unreliable generations can directly affect users and downstream systems (Ma et al., 2026). This concern becomes more pronounced as models are integrated into increasingly open-ended and automated workflows, including emerging agentic applications (Fawzy et al., 2026; Luo et al., 2025). Runtime safeguards (i.e., guardrails) are therefore essential for detecting and mitigating risky model behavior during inference, and have become a critical component of reliable LLM deployment (Group, 2026; Inan et al., 2023; Lin et al., 2026; Qi et al., 2026; Wu et al., 2024). Guardrails monitor interactions between users and models, identify potential risks, and provide signals for interventions such as refusal or regeneration (Cunningham et al., 2026; Han et al., 2024; Sharma et al., 2025). Effective guardrails should account for both sides of an interaction: harmful behavior may originate from a malicious user intent, or emerge within an otherwise benign response as generation unfolds (Zhao et al., 2025a). Moreover, reliable deployment requires monitoring not only harmful content but also potential hallucinations that may undermine the factual reliability of model responses (Huang et al., 2025; Luo et al., 2024).

These requirements motivate a unified guardrail capable of continuously perceiving query intent, response safety, and hallucination risk throughout generation.

Despite their importance, existing LLM guardrails suffer from several key limitations that hinder their practical deployment.

1. **Independent guardrail inference.** Most guardrails operate as separate, independent models (Inan et al., 2023; Zhao et al., 2025a), introducing additional model parameters and deployment complexity. Existing streaming guardrails also compute semantic representations of the generated content from scratch, leaving the LLM’s intermediate representations unexploited. This results in redundant computation and additional communication and synchronization overhead throughout decoding.
2. **Delayed safety signals.** Conventional guardrails typically assess safety only after the complete response has been generated, preventing timely intervention during generation. To amortize the cost of independent guardrail inference, streaming systems often inspect outputs at the chunk level, delaying safety signals until sufficient content has accumulated and limiting fine-grained intervention.
3. **Limited guardrail capacity.** Existing guardrails are often substantially smaller than the LLMs they monitor. As such, there exists a capacity mismatch that may limit their ability to understand complex outputs and make reliable safety judgments, particularly in long-horizon agentic tasks. As LLMs increasingly support million-token contexts, matching their long-context understanding with a separate guardrail becomes increasingly challenging.

Together, these limitations create a fundamental trade-off between detection *effectiveness* and computational *efficiency*, constraining the scaling of guardrail models: improving detection capability generally comes at the cost of higher inference and deployment overhead, while efficiency requirements limit model capacity and monitoring granularity.

To overcome this effectiveness-efficiency trade-off, we draw inspiration from recent findings that *LLM hidden states encode safety-discriminative signals*, yet these signals may fail to prevent harmful generations due to insufficient safety generation ability or jailbreak-induced suppression (Ding et al., 2025; Zhao et al., 2025b). This motivates us to directly reuse the model’s hidden states for intrinsic safety classification. Building on this insight, we introduce SingProbe (i.e., **Safety in Generation Probe**), an intrinsic safety and hallucination detection model integrated into the models. SingProbe directly consumes the hidden states produced during model inference and operates alongside language-model decoding, avoiding repeated text encoding and separate guardrail inference. Within a unified framework, it supports *query intent classification*, *response safety classification*, and *response hallucination detection*. By producing continuously updated predictions throughout generation, SingProbe enables fine-grained, token-level monitoring of safety and hallucination risks with negligible additional inference overhead. For practical deployment, we further integrate SingProbe into

SGLang¹ (Zheng et al., 2024) and vLLM² (Kwon et al., 2023), enabling token generation and guardrail scoring within a unified serving pipeline. This system-level integration removes the need to deploy and coordinate a separate guardrail service, reduces cross-service communication, and allows token-level guardrail signals to share SGLang’s inference infrastructure, thereby simplifying scalable deployment.

To enable systematic evaluation of generation-time safety monitoring, we further introduce **SingStreamBench**, a streaming safety benchmark that explicitly evaluates whether a guardrail can remain inactive on benign prefixes and respond promptly once unsafe content emerges. Across comprehensive evaluations of query-intent classification, response safety, streaming detection, hallucination detection, and benign false-positive robustness, SingProbe achieves competitive or superior performance compared with substantially larger standalone guardrails and specialized hallucination detectors.

Beyond passive detection, we further study SingProbe as a generation-time control signal, including online monitoring under free autoregressive generation and SingProbe-guided constrained decoding, demonstrating that its token-level risk scores can also support safer generation decisions. We additionally extend this paradigm to the medical domain with **SingProbe-Med**, where token-level internal-state signals determine when a medical-risk intervention should be activated. By applying contrastive decoding only to risk-relevant suffixes, SingProbe-Med enables selective, on-demand correction while avoiding continuous intervention on benign generations.

2 Methodology

2.1 Formulation

Let $\mathcal{M}$ denote an autoregressive LLM with L layers. Given a token sequence $\mathbf{x} = (x_1, \dots, x_T)$, the model produces a hidden state $\mathbf{h}_t^{(\ell)} \in \mathbb{R}^d$ for token x_t at layer ℓ . Rather than introducing an additional text encoder, SingProbe directly takes the concatenated hidden states from a selected subset of layers $\mathcal{S} \subseteq \{1, \dots, L\}$ as its input. Abstracting the fusion of the selected hidden states into a lightweight prediction head g_θ , the token-level output is defined as

$$\mathbf{s}_t = g_\theta\left(\left\{\mathbf{h}_t^{(\ell)}\right\}_{\ell \in \mathcal{S}}\right) = \left[\mathbf{s}_t^{\text{query}}, s_t^{\text{unsafe}}, s_t^{\text{hallu}}\right] \in \mathbb{R}^{\mu+1+1}, \quad (1)$$

where $\mathbf{s}_t^{\text{query}} \in \mathbb{R}^\mu$ contains μ fine-grained query-intent scores, while $s_t^{\text{unsafe}} \in \mathbb{R}$ and $s_t^{\text{hallu}} \in \mathbb{R}$ respectively represent the response-unsafety and hallucination scores. All scores are produced at every token position. Because each selected hidden state depends only on the current prefix $x_{\leq t}$, SingProbe provides causally updated predictions throughout generation, enabling token-level streaming detection without an additional encoding pass. We treat these outputs as logits and apply task-specific normalization when probabilities or decisions are required.

Query Intent Classification. Following SingGuard (Group, 2026), we organize query intents into eight categories and use the first eight outputs of SingProbe as their class scores: **(A)** Sexual Content Risk, **(B)** Real-World Crimes & Public Safety, **(C)** Unethical Behavior, **(D)** Cybersecurity & Information Manipulation, **(E)** Agent Safety, **(F)** Politically Sensitive Content, **(G)** Animal Abuse, and **(H)** Safe. The first seven categories cover different types of potentially harmful intent, while Safe indicates that the query does not match any active risk category. A higher class score indicates stronger evidence for the corresponding intent category.

Response Safety and Hallucination Classification. The other output scores track risks in the generated response. The response-unsafety score measures the safety risk of the response prefix generated up to the current token, while the hallucination score estimates the likelihood that the generated content contains hallucinations. Higher values indicate that the response is more likely to be unsafe or contain hallucinations, respectively. Both scores are updated throughout decoding, providing streaming risk signals that can support early intervention.

2.2 Training Objectives

We could jointly train SingProbe for any query- or response-classification task. In our implementation, we train SingProbe on query-intent classification, response-safety classification, and response-hallucination

¹<https://github.com/jinzen-lin/sclang/tree/token-probe-ling3-flash-main>

²<https://github.com/jinzen-lin/vllm/tree/bailing-v3-token-probe>

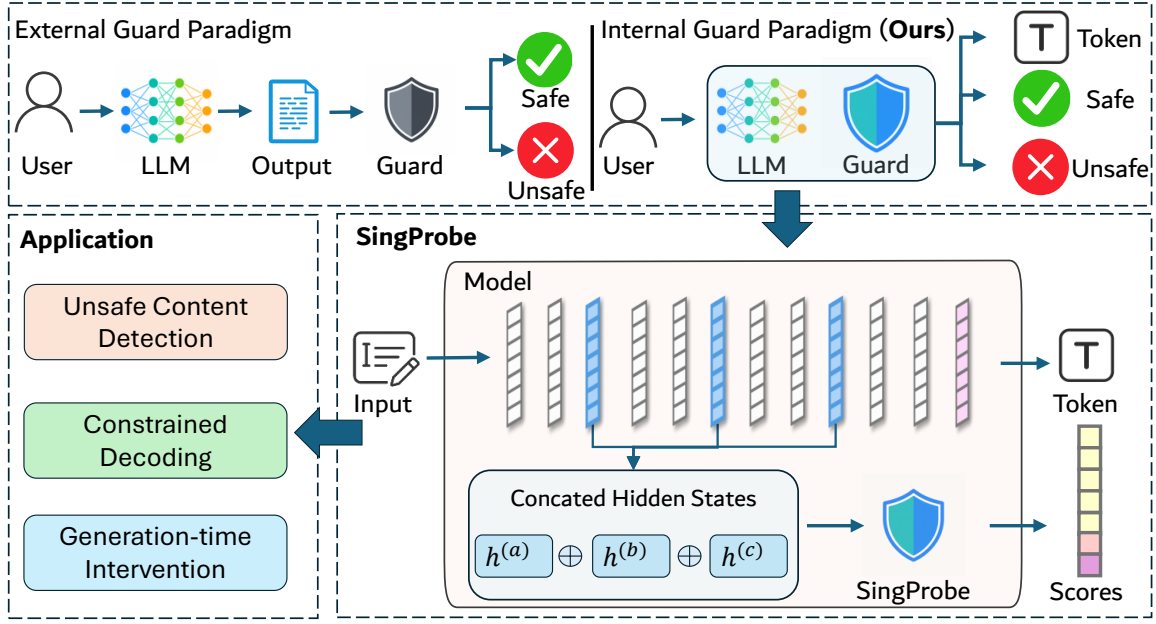


Figure 2: The overview of SingProbe. **Top:** SingProbe replaces the conventional external guard paradigm with an intrinsic guard that operates alongside autoregressive decoding. **Bottom right:** It directly consumes concatenated hidden states from selected base-model layers and produces token-level signals for query intent, response safety, and hallucination risk. **Bottom left:** These streaming signals support applications including unsafe content detection, constrained decoding, and generation-time intervention.

detection. The three objectives are combined as

$$\mathcal{L}_{\text{total}} = w_q \mathcal{L}_{\text{query}} + w_s \mathcal{L}_{\text{safety}} + w_h \mathcal{L}_{\text{hallu}}, \quad (2)$$

where w_q , w_s , and w_h weight the query-intent, response-safety, and response-hallucination objectives, respectively. Rather than keeping these weights fixed, we update them based on exponential moving averages (EMAs) of the corresponding losses, assigning larger weights to tasks with higher recent losses while retaining their configured base weights. Concretely, let $\mathcal{L}_{\tau}^{(t)}$ denote the loss of task $\tau \in \{q, s, h\}$ at step t , and $\tilde{\mathcal{L}}_{\tau}^{(t)}$ its EMA updated with smoothing coefficient α :

$$\tilde{\mathcal{L}}_{\tau}^{(t)} = \alpha \tilde{\mathcal{L}}_{\tau}^{(t-1)} + (1 - \alpha) \mathcal{L}_{\tau}^{(t)}, \quad \tau \in \{q, s, h\}. \quad (3)$$

The per-step weight is then obtained by scaling the base weight w_{τ}^{base} with the ratio of the task’s EMA loss to the mean of those EMAs, $\tilde{\mathcal{L}}^{(t)} = \text{Mean}(\tilde{\mathcal{L}}_{\tau}^{(t)})$:

$$w_{\tau}^{(t)} = w_{\tau}^{\text{base}} \frac{\tilde{\mathcal{L}}_{\tau}^{(t)}}{\tilde{\mathcal{L}}^{(t)}}, \quad \tau \in \{q, s, h\}. \quad (4)$$

Dividing by $\tilde{\mathcal{L}}^{(t)}$ reweights each task in proportion to its recent difficulty (larger recent loss $\Rightarrow$ larger weight), while the three ratios average to 1, so the overall loss magnitude is preserved. We set $\alpha=0.9$ and guard the denominator against zero with a small floor of 10^{-9} . This allows training to focus on the currently more difficult objectives as optimization progresses. The details of the above objectives are introduced as follows.

2.2.1 Query-Intent Objective

The query-intent objective applies binary cross-entropy (BCE) independently to the seven risk categories, allowing multiple risks to coexist, and to the Safe category, which is mutually exclusive with all risk categories. In addition to this label-level constraint, we introduce a soft mutual-exclusion loss that penalizes simultaneously high predictions for Safe and the most probable risk category. This encourages a clear separation between safe and risky queries without restricting co-occurrence among different risk types. The query-intent loss is defined as

$$\mathcal{L}_{\text{query}} = \mathbb{E}_b \left[\frac{1}{|\mathcal{V}_b|} \sum_{t \in \mathcal{V}_b} \left(\frac{1}{\mu - 1} \sum_{k=0}^{\mu-2} \text{BCE}(\ell_{b,t,k}, y_{b,t,k}) + \lambda_{\text{safe}} \text{BCE}(\ell_{b,t,\mu-1}, y_{b,t,\mu-1}) \right. \right. \\ \left. \left. + \lambda_{\text{mutex}} \sigma(\ell_{b,t,\mu-1}) \max_{0 \leq k \leq \mu-2} \sigma(\ell_{b,t,k}) \right) \right]. \quad (5)$$

Here, $\mathcal{V}_b$ denotes the valid supervised positions in sample b ; $\ell_{b,t,k}$ and $y_{b,t,k}$ are the logit and label for category k at position t ; $\sigma(\cdot)$ is the sigmoid function; and λ_{safe} and λ_{mutex} control the Safe loss and mutual-exclusion penalty, respectively.

2.2.2 Response Safety and Hallucination Objectives

Streaming response detection requires learning from every generation prefix, yet available safety and hallucination datasets typically provide only response-level labels rather than fine-grained token-level supervision. Qwen3Guard-Stream (Zhao et al., 2025a) addresses this problem by generating multiple rollouts from each token prefix and estimating its risk from the proportion of unsafe continuations, followed by LLM-based verification. Although this procedure produces fine-grained boundary labels, repeatedly generating and judging multiple continuations for every prefix incurs substantial annotation cost. Constitutional Classifier++ (Cunningham et al., 2026) instead avoids explicit token-level annotation through a softmax-weighted objective that concentrates supervision on tokens with high predicted harmfulness. However, because the weights are determined directly by the current logits, this strategy can be sensitive to poorly calibrated predictions at initialization: spuriously high-scoring tokens may receive disproportionate weights and amplify early training noise.

Based on the above understanding, we therefore adopt an adaptive confidence-weighted aggregator that activates confidence weighting only when token predictions are sufficiently differentiated and otherwise falls back to uniform averaging. The response-safety and response-hallucination objectives are both trained with token-level BCE under this aggregator. Let $r \in \mathcal{R} = \{\text{safety}, \text{hallu}\}$ denote the task and c_r its output dimension, where $c_{\text{safety}} = \mu$ and $c_{\text{hallu}} = \mu + 1$. The corresponding objective is defined as

$$\mathcal{L}_r = \mathbb{E}_b \left[\sum_{t \in \mathcal{V}_b} \alpha_{b,t}^{(r)} \text{BCE}(\ell_{b,t,c_r}, y_{b,t,c_r}) \right], \quad (6)$$

$$\alpha_{b,t}^{(r)} = \text{softmax}_{t \in \mathcal{V}_b} \left(\frac{\beta_r p_{b,t}^{(r)}}{T} \right), \quad \beta_r = \text{clip} \left(\frac{\text{std}_{(b,t) \in \mathcal{V}}(p_{b,t}^{(r)})}{\tau}, 0, 1 \right), \quad p_{b,t}^{(r)} = \sigma(\ell_{b,t,c_r}).$$

Here, $p_{b,t}^{(r)}$ is the predicted risk probability and $\alpha_{b,t}^{(r)}$ is its normalized token weight. The factor β_r controls the strength of confidence weighting according to the dispersion of valid predictions. When predictions are insufficiently differentiated, β_r approaches zero and the aggregator reduces to uniform averaging, preventing early training noise from being amplified. Once the predictions become sufficiently differentiated, β_r increases and the softmax assigns greater weight to tokens with higher predicted risk—unsafety for response safety and hallucination probability for hallucination detection. T is the weighting temperature, and τ controls the sensitivity of the adaptive factor.

This confidence weighting has complementary effects on negative and positive samples. For safe or non-hallucinated responses, high-risk tokens correspond to potential false positives; emphasizing their losses directly suppresses spurious risk peaks. For unsafe or hallucinated responses, tokens in a benign prefix typically receive lower risk scores and are therefore downweighted relative to the later risk-bearing tokens. This prevents the response-level positive label from incorrectly pushing every prefix toward the risky class. Moreover, because the weights are normalized separately within each sample, a benign prefix receives a larger relative weight in a negative response, where no genuinely risky continuation dominates the weighting distribution, than in a positive response containing later high-risk tokens. Consequently, similar benign prefixes receive stronger supervision toward the safe or non-hallucinated class from negative samples while remaining largely unaffected by the coarse positive labels of unsafe or hallucinated samples.

All objectives are computed only at response positions with valid labels. Because token weights are normalized within each sample, longer responses do not dominate training.

2.3 Training Dataset Construction

Training SingProbe requires stable supervision for learning safety- and factuality-related decision boundaries in the hidden-state space of the backbone LLM. We therefore construct a unified corpus for three tasks: *query intent classification*, *response safety classification*, and *response hallucination detection*. The final corpus covers harmful prompts, benign but sensitive prompts, harmful responses, safe refusals, and helpful safe completions. Each example follows a unified format containing a user query, optional dialogue context, and an optional assistant response. Query-side and response-side labels are maintained separately, allowing the same data source to support prompt screening, response monitoring, and joint query–response classification. The corpus is constructed through open-source data curation, policy-grounded synthesis, and multi-stage quality control.

Open-Source Data Curation. We use only the official training splits of publicly available safety benchmarks and training datasets. Because different sources adopt different annotation protocols, risk granularities, and decision thresholds, directly merging their labels would introduce inconsistent supervision. We therefore normalize all safety examples under the fine-grained risk taxonomy introduced in this work.

For datasets with existing annotations, the original labels are first mapped to our taxonomy using predefined category correspondences and then independently verified by an LLM judge under our safety definitions. Samples with consistent labels are retained, whereas ambiguous cases, including potential safe/unsafe polarity flips, are moved to an unlabeled hard-case pool. For unlabeled data and hard cases, multiple open-source LLMs, including Qwen3.5-397B-A17B (Qwen Team, 2026) and Kimi-K2.6 (Moonshot AI, 2026), independently annotate each example. We combine self-consistency across repeated predictions from the same model with agreement across different models. Each accepted sample must pass both a binary safe/unsafe consistency check and a fine-grained category check. Samples failing the binary check are discarded, while samples with unresolved category disagreement are retained only after targeted review.

For hallucination supervision, we additionally incorporate the training splits of public factuality and hallucination datasets, labeling correct answers and honest refusals as *non-hallucinated* and incorrect, fabricated, or unsupported answers as *hallucinated*.

Policy-Grounded Safety Data Synthesis. Open-source safety data are typically long-tailed and provide limited coverage of emerging or low-frequency risks. We therefore supplement them with policy-grounded synthetic data. For each risk category, an internally red-teamed jailbreak model generates harmful prompts, while an aligned model produces topically similar but benign counterparts. These contrastive pairs share surface vocabulary but differ in their underlying intent, discouraging SingProbe from relying on lexical shortcuts for the safety task.

We further generate multiple responses for each prompt. Unsafe prompts are paired with both harmful responses and policy-compliant refusals or safe alternatives, while benign prompts are paired with helpful and compliant responses. This construction prevents response safety from being inferred directly from the prompt label and encourages SingProbe to track the evolving response representation. We additionally synthesize multi-category and multilingual examples to improve coverage of compositional and cross-lingual risks. All synthetic samples are independently re-evaluated by an LLM that is not involved in generation, and samples whose predicted labels disagree with the intended labels are removed.

Quality Control and Training Supervision. We remove duplicate and near-duplicate examples, balance safe and unsafe instances across major risk categories, and retain high-confidence boundary cases that distinguish harmful intent from benign discussions of sensitive topics. During training, the curated examples are processed by the backbone LLM to obtain the hidden states used by SingProbe. Query labels are aligned with representations at the query boundary, while response-safety and hallucination labels supervise representations produced along the response trajectory. This formulation enables SingProbe to jointly monitor user intent, response safety, and factual reliability without requiring an additional text encoder or a separately deployed guardrail model.

3 SingStreamBench: Benchmarking Streaming Safety Detection

3.1 Motivation

Existing safety benchmarks are primarily designed for post-hoc classification (Ghosh et al., 2025; Han et al., 2024; Ji et al., 2023; Mazeika et al., 2024) and provide only a single response-level label. Such labels indicate whether a completed response is unsafe, but do not reveal when unsafe content first emerges during generation. Consequently, they cannot adequately evaluate whether a streaming guardrail remains silent on

Table 1: Tiered construction of SingStreamBench.

Tier	Query	Response structure	Primary purpose
0	Original query	Original response	Establish performance on ordinary query-response samples.
1	Unsafe query	Safe refusal	Test whether the guardrail spuriously reacts to unsafe intent in the query despite a safe response.
2	Original query	Safe prefix + safe continuation	Test whether the guardrail remains silent when a safe prefix is continued safely.
3	Original query	Complex safe prefix + unsafe response	Test whether background information or disclaimers delay or otherwise interfere with detection at the true unsafe onset.
4	Background context + original query	Original unsafe response	Test robustness to complex query-side context and whether the guardrail attributes response risk correctly.
5	Multi-question query	Safe answers/prefix + unsafe response	Jointly test complex query context and a substantial safe response prefix before unsafe content appears.

benign prefixes, detects the actual onset of harmful content, or intervenes with minimal delay.

FineHarm (Li et al., 2025b) takes an important step toward fine-grained evaluation by providing token-level harmfulness annotations. Its annotations are constructed heuristically: responses are first labeled at the sentence level using an LLM, after which content words in harmful sentences are marked as harmful and their labels are mapped to the corresponding tokens. However, our manual inspection reveals substantial noise in these fine-grained labels. More fundamentally, harmfulness is often expressed compositionally and depends on surrounding context, making it difficult to assign a stable safety label to an individual token in isolation. These issues limit the reliability of token-level labels as ground truth for evaluating the precise onset of unsafe content.

Even when fine-grained labels can be obtained, the composition of existing safety datasets introduces another limitation. Most unsafe responses begin producing harmful content near the start of the response, resulting in a strong positional bias and few examples containing a substantial safe prefix before the unsafe segment. A classifier may therefore appear effective by reacting to the unsafe query or early lexical cues, without demonstrating that it can remain inactive throughout benign content and trigger only when harmful content actually appears.

Based on these observations, we construct a manually verified fine-grained benchmark, **SingStreamBench**, and a larger-scale version, **SingStreamBench-Full**, to evaluate both the accuracy and timeliness of streaming safety detection. Instead of assigning intrinsic labels to isolated tokens, we segment each response into sentences and annotate its cumulative sentence-level prefixes to identify a semantically meaningful unsafe onset boundary. We further construct controlled safe-to-unsafe transitions and fully safe counterexamples, organized into six tiers that vary the complexity and safety composition of both queries and response prefixes. This design enables explicit evaluation of whether a guardrail remains silent before risk emerges, detects the true unsafe segment promptly, and avoids relying on spurious query or positional cues.

3.2 Benchmark Design

Each benchmark sample consists of a query $\mathbf{q}$ and a response $\mathbf{r}$ that can be conceptually decomposed as

$$\mathbf{r} = \mathbf{r}^{\text{safe}} \oplus \mathbf{r}^{\text{target}}, \quad (7)$$

where $\mathbf{r}^{\text{safe}}$ is an optional safe prefix and $\mathbf{r}^{\text{target}}$ is the continuation whose safety status determines the response label. The central evaluation target is the transition around the beginning of $\mathbf{r}^{\text{target}}$: a reliable streaming guardrail should remain silent throughout the safe prefix and react promptly once unsafe content emerges. For fully safe responses, it should remain silent throughout generation.

We organize the benchmark into six tiers, as summarized in Table 1. The tiers progressively test spurious associations with the query or safe prefix and robustness to increasingly complex contexts.

Tiers 1 and 2 primarily diagnose false associations. Tier 1 tests whether an unsafe query alone causes the response stream to be flagged even when the model safely refuses, while Tier 2 tests whether a safe prefix itself induces false alarms when followed by a safe continuation. Tiers 3–5 introduce more complex contexts.

Tier 3 inserts background knowledge or a disclaimer before the original unsafe response; Tier 4 prepends query-related background text to the query; and Tier 5 combines one unsafe question with multiple safe questions, producing a multi-question context in which safe content precedes the unsafe segment. Together, these tiers distinguish genuine localization of response risk from shortcuts based on query harmfulness, response position, or superficial lexical cues.

3.3 Construction Pipeline

The benchmark is constructed through the following four-stage pipeline.

1. **Seed sampling and verification.** We sample query–response pairs from existing safety datasets and re-evaluate whether each response-level label is correct. Three judge models, Qwen3-235B-A22B (Yang et al., 2025), GLM-5.2 (Zeng et al., 2026), and Kimi-K2.6 (Moonshot AI, 2026), independently verify each sample, and only samples unanimously accepted by all three models are retained. This step removes mislabeled and ambiguous seed responses.
2. **Tiered data augmentation.** Starting from a verified unsafe query–response pair, we construct four types of augmented samples. For Tier 1, a safe refusal is generated for the unsafe query. For Tier 3, the model generates safe background information or a disclaimer that is prepended to the original unsafe response. For Tier 4, query-related background text is generated and prepended to the original query. For Tier 5, one unsafe question is combined with one to three sampled safe questions using a multi-question template such as “Please answer the following questions: 1. ...; 2. ...”. For each generation operation, one of the three models above is selected at random to increase stylistic and model diversity.
3. **Sentence-boundary prefix annotation.** Each response is first segmented into sentences $\mathbf{u}_1, \dots, \mathbf{u}_m$. We then construct cumulative prefixes $\mathbf{P}_j = \mathbf{u}_1 \oplus \dots \oplus \mathbf{u}_j$ and use Kimi-K2.6 to assign a Safe or Unsafe label to each prefix $\mathbf{P}_j$, rather than labeling each sentence in isolation. For an unsafe response, annotation proceeds sequentially and stops once the first unsafe prefix is identified; the starting position of its newly added sentence is recorded as `Unsafe_Start_Index`. All preceding prefixes are treated as safe. For a safe response, every sentence-level prefix is verified as safe. This procedure directly identifies the safe-prefix boundary required for streaming evaluation while avoiding token-level semantic ambiguity.
4. **Safe-continuation augmentation.** To construct Tier 2, we sample examples with a non-empty safe prefix and ask a model to continue generation safely from the corresponding query and prefix. The resulting fully safe responses test whether a guardrail can remain inactive after observing prefixes that may otherwise also occur before unsafe continuations.

3.4 Dataset Variants and Statistics

The construction pipeline produces two benchmark variants. **SingStreamBench** is the high-quality core benchmark containing 210 samples that have undergone manual verification. **SingStreamBench-Full** is the larger-scale variant containing 2,428 constructed samples, providing broader coverage across tiers and safety scenarios. The original queries and responses are partly from BeaverTails (Ji et al., 2023), PKU-SafeRLHF (Ji et al., 2024), WildGuard (Han et al., 2024), XSTest (Röttger et al., 2023), and expguard (Choi et al., 2026). The manually verified core set supports reliable comparison of streaming detection accuracy and latency, while the full set enables evaluation at a larger scale.

4 Evaluation

4.1 Experimental Settings

Safety Guard Baselines. We compare SingProbe against two proprietary LLMs and 12 open-source guardrail configurations. The proprietary baselines are Gemini 3 Pro (Google, 2025) and GPT-5.1 (OpenAI, 2025). The open-source baselines include YuFeng-XGuard-Reason-0.6B/8B (Lin et al., 2026), Llama Guard 3 (Inan et al., 2023), WildGuard (Han et al., 2024), GraniteGuardian (Research, 2024), ShieldGemma-9B (Zeng et al., 2024), and Qwen3Guard series (Zhao et al., 2025a). For Qwen3Guard, we evaluate both loose and strict modes, which respectively treat the Controversial label as safe and unsafe. Some of the baselines’ evaluation results are from Group (2026).

Hallucination Detection Baselines. We compare SingProbe with four representative hidden-state-based hallucination detectors: DRIFT (Bhatnagar et al., 2026), HaMI (Niu et al., 2025), and SAPLMA (Azaria & Mitchell, 2023). These methods cover lightweight probing, adaptive token selection, and training-free NTK-based scoring. For SAPLMA, we evaluate both the raw-input and response-formatted variants, denoted SAPLMA-raw and SAPLMA-response, respectively.

Table 2: Query intent classification evaluation results. The values in the table are F1 scores.

Model	Aegis2	XGuard Test	OpenAI Moderation	XSTest	HarmBench	ExpGuardTest	Avg.
Gemini3-Pro	0.7401	0.8426	0.7545	0.8962	0.8615	0.7980	0.8155
GPT-5.1	0.8142	0.8815	0.8024	0.9169	0.9392	0.8554	0.8683
YuFeng-XGuard-Reason-0.6B	0.8625	0.9126	0.7107	0.9179	0.9250	0.7476	0.8461
YuFeng-XGuard-Reason-8B	0.8541	0.9266	0.7187	0.9461	0.9466	0.8361	0.8714
Llama Guard 3	0.7635	0.7909	0.7904	0.8852	0.8544	0.7077	0.7987
WildGuard	0.8076	0.8237	0.7252	0.9501	0.9288	0.8438	0.8465
GraniteGuardian	0.7138	0.5777	0.5684	0.7269	0.9045	0.9115	0.7338
ShieldGemma-9B	0.7588	0.6885	0.8143	0.8293	0.6441	0.5340	0.7115
Qwen3Guard-Gen-4B-loose	0.8054	0.7952	0.7897	0.8674	0.8311	0.7139	0.8005
Qwen3Guard-Gen-4B-strict	0.8499	0.8789	0.6764	0.8791	0.9260	0.8312	0.8402
Qwen3Guard-Gen-8B-loose	0.8199	0.8197	0.7958	0.8815	0.8401	0.6953	0.8087
Qwen3Guard-Gen-8B-strict	0.8498	0.8797	0.6804	0.8796	0.9326	0.8297	0.8420
Qwen3Guard-Stream-8B-loose	0.8091	0.8282	0.8023	0.9016	0.8390	0.7519	0.8220
Qwen3Guard-Stream-8B-strict	0.8602	0.8752	0.7386	0.9223	0.9231	0.8416	0.8602
SingProbe-Ling-3.0-tiny	0.8488	0.8684	0.7510	0.8926	0.9035	0.8724	0.8561
SingProbe-Ling-3.0-flash	0.8481	0.8795	0.7703	0.9012	0.9231	0.8825	0.8674

Implementation Details of SingProbe. In our experiments, we implement a unified SingProbe architecture based on a query-residual multi-head attention (MHA) block. Specifically, the block computes queries Q , keys K , and values V and produces the attention output O , and finally outputs $\text{Norm}(Q + O)$, followed by a linear classifier that produces the guardrail scores. As the default setting, we use hidden states from three layers of the base model as input to SingProbe, with the layers evenly selected from the shallow, middle, and deep portions of the model.

4.2 Query Classification Evaluation

Table 2 reports the query-intent classification results across six benchmarks, including Aegis2 (Ghosh et al., 2025), XGuard (Lin et al., 2026), OpenAI Moderation (Markov et al., 2023), XSTest (Röttger et al., 2023), HarmBench (Mazeika et al., 2024), and ExpGuard (Choi et al., 2026). For each sample, we feed the query to the base model and let it generate a response with a maximum length of 512. SingProbe produces query-intent scores at every token position in the generated response. We then average these scores across response positions and use the resulting eight-dimensional score vector as the final query-intent prediction. The best configuration, SingProbe with Ling-3.0-flash, achieves an average F1 of 0.8674, ranking closely behind YuFeng-XGuard-Reason-8B (0.8714) and GPT-5.1 (0.8683). It also outperforms all evaluated Qwen3Guard configurations, whose best average F1 is 0.8602, as well as other widely used guardrails such as WildGuard and Llama Guard 3. These results show that an intrinsic lightweight head can provide query-safety classification performance comparable to substantially larger standalone guardrail models and frontier proprietary LLMs.

The results further demonstrate consistent benefits from scaling the base model. Replacing Ling-3.0-tiny with Ling-3.0-flash improves the average F1 of SingProbe from 0.8561 to 0.8674, indicating that stronger base-model hidden states provide richer query-safety signals for the lightweight head. SingProbe is competitive with the strongest standalone guardrails across the six benchmarks and achieves the best F1 on ExpGuardTest (0.8825), suggesting that the base model’s hidden states already contain strong query-safety signals that can be effectively extracted by a lightweight temporal head.

4.3 Response Safety Evaluation

We evaluate response safety from three complementary dimensions: *response-level classification*, which measures whether a guardrail correctly identifies the overall safety of a completed response; *streaming detection*, which evaluates both response-level discrimination and token-level risk localization throughout generation; and *false-positive robustness*, which measures whether the guardrail remains silent on nominally benign queries and freely generated responses.

Response-level Classification. We first evaluate response safety under teacher forcing by feeding each query together with its labeled response. We evaluate 8 benchmarks including BeaverTails (Ji et al., 2023), PKU-SafeRLHF (Ji et al., 2024), Aegis2 (Ghosh et al., 2025), WildGuard (Han et al., 2024), XGuard (Lin et al.,

Table 3: Response safety classification evaluation results. The values in the table are F1 scores.

Model	BeaverTails	PKU-SafeRLHF	Aegis2	WildGuard	XGuard Test	HarmBench	XSTest	ExpGuardTest	Avg.
Gemini3-Pro	0.7960	0.8605	0.7667	0.5478	0.7172	0.7632	0.5984	0.8921	0.7427
GPT-5.1	0.8042	0.8880	0.7861	0.6723	0.7292	0.7812	0.6667	0.9151	0.7804
YuFeng-XGuard-Reason-0.6B	0.8624	0.9181	0.8253	0.7471	0.8547	0.8759	0.8000	0.9125	0.8495
YuFeng-XGuard-Reason-8B	0.8571	0.9144	0.8303	0.7350	0.8556	0.8672	0.8387	0.9121	0.8513
Llama Guard 3	0.6778	0.8908	0.6110	0.7030	0.6667	0.8668	0.9041	0.8419	0.7703
WildGuard	0.7789	0.8189	0.8644	0.5194	0.6265	0.6976	0.5929	0.8998	0.7248
GraniteGuardian	0.7291	0.6869	0.6324	0.2861	0.6837	0.6667	0.2977	0.8868	0.6087
ShieldGemma-9B	0.6962	0.8214	0.6765	0.5316	0.6142	0.6463	0.8652	0.4014	0.6566
Qwen3Guard-Gen-4B-loose	0.8455	0.9317	0.7874	0.7850	0.7254	0.8870	0.9172	0.8230	0.8378
Qwen3Guard-Gen-4B-strict	0.8581	0.9060	0.8351	0.7750	0.7953	0.9061	0.8721	0.8986	0.8558
Qwen3Guard-Gen-8B-loose	0.8493	0.9308	0.8309	0.7881	0.7542	0.8816	0.9103	0.8455	0.8488
Qwen3Guard-Gen-8B-strict	0.8669	0.9043	0.8407	0.7778	0.7906	0.8907	0.8982	0.9140	0.8604
Qwen3Guard-Stream-8B-loose	0.8560	0.9076	0.8203	0.7866	0.7651	0.8746	0.8539	0.8973	0.8452
Qwen3Guard-Stream-8B-strict	0.8570	0.9019	0.8208	0.7713	0.7697	0.8882	0.8556	0.9244	0.8486
SingProbe-Ling-3.0-tiny	0.8537	0.9120	0.8211	0.7683	0.8122	0.8645	0.8588	0.9159	0.8508
SingProbe-Ling-3.0-flash	0.8433	0.9301	0.8333	0.8000	0.8588	0.8995	0.8929	0.9247	0.8728

Table 4: Streaming safety evaluation results. Response-level AUC (R-AUC) evaluates response-wise discrimination using the maximum response-safety score over all token positions as the response risk score. Token-level AUC (T-AUC) evaluates token-wise discrimination by comparing each token’s response-safety score with its corresponding fine-grained label.

Model	SingStreamBench		SingStreamBench-Full		FineHarm		Avg.	
	R-AUC	T-AUC	R-AUC	T-AUC	R-AUC	T-AUC	R-AUC	T-AUC
Qwen3Guard-Stream-4B-loose	0.9594	0.8849	0.9194	0.8389	0.9874	0.9456	0.9554	0.8898
Qwen3Guard-Stream-4B-strict	0.9669	0.8896	0.9313	0.8528	0.9938	0.9540	0.9640	0.8988
Qwen3Guard-Stream-8B-loose	0.9591	0.8678	0.9239	0.8283	0.9905	0.9504	0.9578	0.8822
Qwen3Guard-Stream-8B-strict	0.9616	0.8684	0.9370	0.8447	0.9933	0.9549	0.9640	0.8893
SingProbe-Ling-3.0-tiny	0.9935	0.9529	0.9775	0.9340	0.9954	0.9568	0.9888	0.9479
SingProbe-Ling-3.0-flash	0.9922	0.9599	0.9778	0.9294	0.9961	0.9549	0.9887	0.9481

2026), HarmBench (Mazeika et al., 2024), XSTest (Röttger et al., 2023), and ExpGuard (Choi et al., 2026). SingProbe produces a response-safety score at every response position, and we use the maximum score over the entire response as the final response-level risk score. As shown in Table 3, SingProbe with Ling-3.0-flash achieves the best average F1 of 0.8728, outperforming the strongest standalone baseline, Qwen3Guard-Gen-8B-strict, by 1.24 points. SingProbe with Ling-3.0-tiny remains competitive at 0.8508, and scaling from Ling-3.0-tiny to Ling-3.0-flash yields consistent gains across nearly all benchmarks. In particular, SingProbe with Ling-3.0-flash achieves the best F1 on WildGuard (0.8000), XGuard Test (0.8588), and ExpGuardTest (0.9247).

Streaming Safety Detection. Table 4 evaluates detection throughout generation on SingStreamBench, SingStreamBench-Full, and FineHarm (Li et al., 2025b). Response-level AUC (R-AUC) is computed using the maximum response-safety score over all token positions and takes the response-level label as the ground-truth label, whereas token-level AUC (T-AUC) directly evaluates the score at each token position against the corresponding fine-grained annotation. The tokens in the annotated safe prefix are labeled as 0. All SingProbe configurations outperform the Qwen3Guard-Stream baselines in both average R-AUC and T-AUC. The best configuration, SingProbe with Ling-3.0-tiny, achieves average R-AUC and T-AUC scores of 0.9888 and 0.9479, improving over the strongest Qwen3Guard-Stream results by 2.48% and 4.91%, respectively. SingProbe with Ling-3.0-flash yields nearly identical average scores of 0.9887 R-AUC and 0.9481 T-AUC. The improvements are especially pronounced on SingStreamBench and SingStreamBench-Full, while performance remains comparable on FineHarm, demonstrating stronger localization of unsafe content under long or complex safe prefixes.

False-positive Evaluation. We additionally evaluate false alarms under free generation using five nominally benign datasets. These datasets are MATH (Hendrycks et al., 2021), databricks-dolly-15k (Conover et al., 2023), GSM8K (Cobbe et al., 2021), MBPP (Austin et al., 2021), and OpenBookQA (Mihaylov et al., 2018).

Table 5: The false positive rate (FPR, in %) of guardrail models on benign data. Each cell shows query/response FPR (lower is better). For Qwen3Guard models we report both the *Loose* (Unsafe only) and *Strict* (Unsafe+Controversial) judgments.

Model	MATH	databricks-dolly-15k	GSM8K	MBPP	openbookqa	Avg.
Llama-Guard-3-8B	0.00/0.00	0.25/0.16	0.00/0.00	0.00/0.00	0.05/0.20	0.06/0.07
YuFeng-XGuard-Reason-8B	0.01/0.01	0.38/0.50	0.15/0.08	0.00/0.00	0.62/0.55	0.23/0.23
Granite-Guardian-4.1-8B	0.00/0.00	0.37/0.34	0.00/0.00	0.00/0.42	2.86/0.97	0.65/0.35
ShieldGemma-9B	0.00/0.00	0.27/0.22	0.00/0.00	0.00/0.00	0.89/0.50	0.23/0.15
WildGuard	0.00/0.00	0.57/0.40	0.00/0.00	0.00/0.00	1.02/0.59	0.32/0.20
Qwen3Guard-Gen-4B-Loose	0.00/0.00	0.13/0.25	0.00/0.00	0.00/0.00	0.02/0.08	0.03/0.07
Qwen3Guard-Gen-4B-Strict	0.00/0.00	0.63/0.76	0.00/0.00	0.00/0.00	0.29/0.40	0.18/0.23
Qwen3Guard-Gen-8B-Loose	0.00/0.00	0.09/0.20	0.00/0.00	0.00/0.00	0.00/0.10	0.02/0.06
Qwen3Guard-Gen-8B-Strict	0.00/0.00	0.50/0.74	0.00/0.00	0.00/0.00	0.29/0.59	0.16/0.27
Qwen3Guard-Stream-4B-Loose	0.00/0.00	0.15/0.24	0.00/0.00	0.00/0.00	0.05/0.05	0.04/0.06
Qwen3Guard-Stream-4B-Strict	0.00/0.00	0.76/0.28	0.00/0.00	0.00/0.00	0.32/0.05	0.22/0.07
Qwen3Guard-Stream-8B-Loose	0.00/0.00	0.12/0.23	0.00/0.00	0.00/0.00	0.00/0.02	0.02/0.05
Qwen3Guard-Stream-8B-Strict	0.00/0.00	0.37/0.28	0.00/0.00	0.00/0.00	0.22/0.02	0.12/0.06
SingProbe-Ling-3.0-tiny	0.00/0.00	0.41/0.14	0.00/0.00	0.00/0.00	0.87/0.05	0.32/0.07
SingProbe-Ling-3.0-flash	0.00/0.00	0.25/0.07	0.00/0.00	0.00/0.00	0.13/0.00	0.13/0.03

Table 6: Hallucination detection — **response-level AUC** across 6 offline hallucination benchmarks. For each response, the Guard output is MAX-pooled across the response token mask, then the ROC AUC is computed sample-vs-sample (threshold-independent). **Macro** is the equal-weight mean of the per-benchmark AUCs.

Model	FactCHD	FaithDial	FAVA	RAGTruth	Shroom	WikiBio	Avg.
Ling-3.0-tiny							
DRIFT	0.7566	0.9030	0.5871	0.6771	0.6736	0.8477	0.7408
HaMI	0.6722	0.8230	0.5256	0.6168	0.6675	0.8381	0.6905
SAPLMA-raw	0.7389	0.8381	0.5516	0.6375	0.6460	0.6294	0.6736
SAPLMA-response	0.7018	0.8480	0.5756	0.6984	0.6832	0.9133	0.7367
SingProbe	0.7819	0.9529	0.5802	0.6980	0.6946	0.9515	0.7765
Ling-3.0-flash							
DRIFT	0.7972	0.9115	0.6577	0.7657	0.7266	0.9414	0.8000
HaMI	0.7571	0.8542	0.5801	0.6253	0.6899	0.9200	0.7378
SAPLMA-raw	0.7904	0.8601	0.5112	0.6845	0.6743	0.7828	0.7172
SAPLMA-response	0.7542	0.8776	0.5835	0.7538	0.7050	0.9701	0.7740
SingProbe	0.8308	0.9498	0.5996	0.7282	0.7314	0.9674	0.8012

Each model receives a query and generates its own response, during which we collect both query-intent and response-safety scores; the final predictions use the same aggregation rules described above. Table 5 reports query / response FPR in each cell. SingProbe maintains a low average response FPR, with 0.07% on Ling-3.0-tiny and 0.03% on Ling-3.0-flash, on par with the strongest Qwen3Guard-Stream baselines. We note that these datasets are not exhaustively safety-audited and may contain a small number of genuinely unsafe queries or responses. The reported values should therefore be interpreted as apparent FPRs and may slightly overestimate the true false-positive rates.

4.4 Response Hallucination Evaluation

We evaluate response-level hallucination detection on six offline benchmarks (FactCHD (Chen et al., 2023), FaithDial (Dziri et al., 2022), FAVA (Mishra et al., 2024a), RAGTruth (Niu et al., 2024), Shroom (Mickus et al., 2024), and WikiBio (Manakul et al., 2023)). Given a query-response pair, SingProbe produces a hallucination score at every response token under teacher forcing. We use the maximum score over all response positions as the response-level risk score and compute ROC AUC across samples. Table 6 reports the per-benchmark results and their macro average.

Table 7: Relative latency overhead of SingProbe under different concurrency levels, reported as mean $\pm$ standard deviation over 8 measurement rounds. TTFT denotes time to first token, and ITL denotes inter-token latency. *Decode Probe* runs SingProbe only during decoding, while *Prefill-enabled Probe* additionally runs SingProbe during the prefill stage. Positive values indicate additional overhead relative to the base model without SingProbe.

Concurrency	Decode Probe		Prefill-enabled Probe	
	TTFT ($\Delta\%$)	ITL ($\Delta\%$)	TTFT ($\Delta\%$)	ITL ($\Delta\%$)
1	$+0.68 \pm 0.22$	$+0.18 \pm 0.15$	$+0.89 \pm 0.29$	$+0.18 \pm 0.15$
2	-1.45 ± 4.17	$+0.33 \pm 0.12$	$+2.25 \pm 4.86$	$+0.33 \pm 0.12$
4	-1.91 ± 5.97	$+0.30 \pm 0.12$	$+1.85 \pm 7.77$	$+0.38 \pm 0.10$
8	$+0.39 \pm 1.84$	$+0.26 \pm 0.09$	$+1.01 \pm 0.53$	$+0.21 \pm 0.12$
16	$+0.06 \pm 3.57$	$+0.42 \pm 0.09$	$+0.86 \pm 1.15$	$+0.39 \pm 0.12$
32	$+1.14 \pm 0.93$	$+0.45 \pm 0.18$	$+1.34 \pm 0.37$	$+0.47 \pm 0.15$

SingProbe achieves strong and consistent performance across both base models. On Ling-3.0-tiny, SingProbe obtains the best average AUC of 0.7765, outperforming the strongest baseline, DRIFT, by 3.57 points, and achieves the best per-benchmark AUC on FactCHD (0.7819), FaithDial (0.9529), and WikiBio (0.9515). Scaling to Ling-3.0-flash further improves SingProbe to 0.8012, outperforming the best baseline DRIFT (0.8000). These results indicate that the hidden states of the base model provide effective signals for hallucination detection and that their quality improves with model scale, enabling a lightweight intrinsic head to be competitive with or surpass specialized hallucination detectors.

4.5 Overhead Evaluation

We evaluate the inference overhead introduced by SingProbe (with the unified architecture on Ling-3.0-flash) under a controlled serving-load protocol. Each request uses an input length of 8,192 tokens and an output length of 1,024 tokens, and we vary the concurrency from 1 to 32. At each concurrency level, we submit a fixed number of prompts per round (24 for concurrency 1, 2, and 4; 48 for concurrency 8; and 96 for concurrency 16 and 32) and repeat the measurement over 8 independent rounds. Within each round, we compute the median TTFT and ITL, pair each SingProbe-enabled configuration with the base-model-only baseline of the same round, and report the mean and standard deviation of the paired relative differences across rounds. We consider two deployment modes of SingProbe: *Decode Probe*, which runs the probe only during autoregressive decoding, and *Prefill-enabled Probe*, which additionally invokes the probe during the prefill stage.

As shown in Table 7, the additional latency introduced by SingProbe remains small across all concurrency levels. For the *Decode Probe* configuration, the ITL overhead is consistently below 0.5%, ranging from $+0.18\%$ at concurrency 1 to $+0.45\%$ at concurrency 32. The TTFT differences are within measurement noise: they are typically within $\pm 1.5\%$ and even show slightly negative point estimates at concurrency 2 and 4, with standard deviations that comfortably span zero. This is expected because the decode-only probe does not participate in the prefill phase and therefore should not change TTFT in a systematic way. Enabling the probe during prefill (*Prefill-enabled Probe*) adds a small but consistently positive TTFT overhead, ranging from $+0.86\%$ to $+2.25\%$, while the ITL overhead remains essentially unchanged and below 0.5% ($+0.18\%$ to $+0.47\%$). Overall, the relative overhead does not grow with serving concurrency, and even in the more expensive prefill-enabled setting the additional cost stays around 1–2% on TTFT and well under 0.5% on ITL. These results demonstrate that continuously extracting guardrail signals from the base model’s hidden states introduces little additional cost to either first-token latency or token-by-token decoding, and that this overhead remains stable under increasing serving load. This efficiency supports the practical deployment of SingProbe as an intrinsic streaming guardrail without requiring a separate guardrail inference service.

4.6 Ablation Study

4.6.1 Effect of the Number of Tapped Layers

We ablate the number of hidden layers tapped from the frozen base model, varying $L \in \{1, 2, 3, 5, 8\}$ while keeping the Guard architecture and training recipe fixed. As shown in Figure 3, safety detection is generally robust to the number of tapped layers. Query AUC remains within a narrow range of 0.9560–0.9579, while Response AUC varies from 0.9648 to 0.9684. Notably, using only the final layer already yields strong performance, achieving a Response AUC of 0.9648 and a streaming response AUC of 0.9767. Incorporating

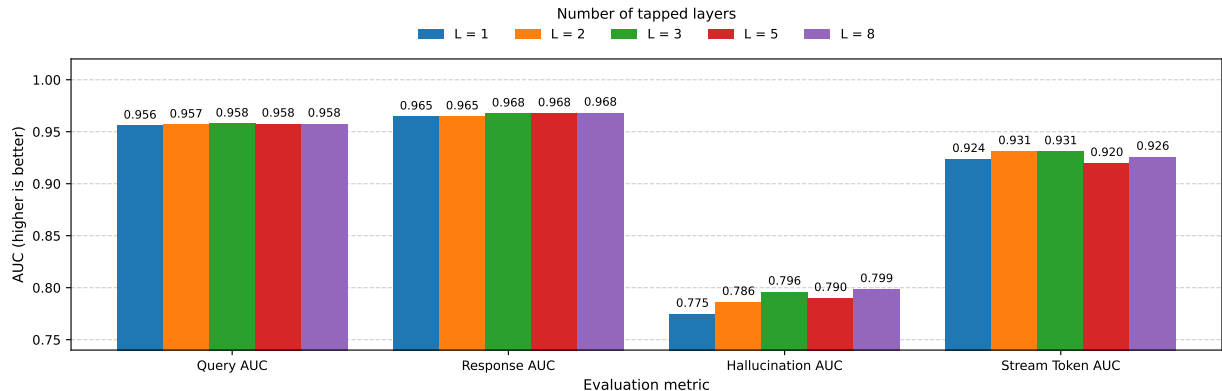


Figure 3: The ablation study of the number of tapped base-model layers.

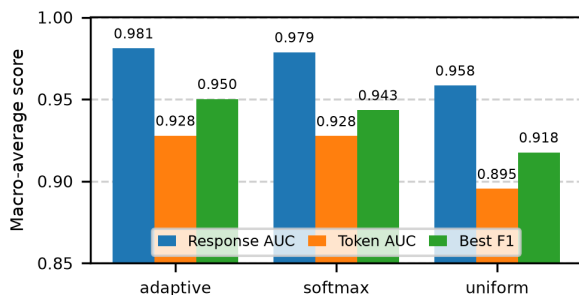


Figure 4: The ablation study of the token weighting techniques.

intermediate-layer representations nevertheless provides modest improvements: compared with $L=1$, the $L=3$ configuration improves Response AUC from 0.9648 to 0.9677 and streaming response AUC from 0.9767 to 0.9823. Hallucination detection benefits more clearly from multi-layer features, with AUC increasing from 0.7750 at $L=1$ to 0.7963 at $L=3$ and reaching 0.7988 at $L=8$. This suggests that hallucination detection may require information that is less completely captured by the final-layer representation. Across all configurations, the benign false-positive rate remains at or below 0.02%, indicating that tapping additional layers does not noticeably increase over-triggering on benign inputs. However, tapping more layers increases the input dimensionality of the Guard, leading to a larger parameter count, higher memory consumption, and additional inference latency. Since the performance gains become marginal beyond $L=3$ while these deployment costs continue to grow, we use three tapped layers by default to achieve a favorable effectiveness-efficiency trade-off.

4.6.2 Effect of the Token Weighting Technique

We ablate the token weighting technique used for the response-safety and hallucination objectives, comparing our adaptive confidence weighting with fixed softmax weighting and uniform averaging while keeping the model architecture and training recipe unchanged. As shown in Figure 4, token-aware weighting consistently outperforms uniform averaging. The uniform variant achieves macro-average Response AUC, Token AUC, and Best F1 scores of 0.958, 0.895, and 0.918, respectively. Applying fixed softmax weighting substantially improves these results to 0.979, 0.928, and 0.943, demonstrating the benefit of assigning greater supervision to more informative, high-risk token positions rather than treating all response positions equally. Our adaptive weighting further improves Response AUC to 0.981 and Best F1 to 0.950, while maintaining the same Token AUC of 0.928. Compared with uniform averaging, this corresponds to gains of 2.3, 3.3, and 3.2 points on the three metrics, respectively. These results suggest that confidence-based token weighting is important for learning from coarse response-level supervision, as uniform weighting can dilute the contribution of risk-bearing tokens with large numbers of benign-prefix tokens. The additional improvement over fixed softmax weighting further supports adapting the weighting strength to the confidence of current predictions: when token scores are insufficiently differentiated, reducing the weighting strength avoids over-emphasizing potentially noisy predictions, while stronger weighting becomes beneficial once meaningful token-level

Table 8: Online response safety detection under free autoregressive generation. For each evaluated detector, pseudo-ground-truth labels are obtained by majority vote of the other three detectors under a leave-one-out protocol. The SingProbe detector is evaluated with its bias-calibrated checkpoint at the canonical 0.5 decision threshold. Values report Accuracy (Acc) and F1. The best result within each backbone is highlighted in bold.

Method	Harmbench		WildGuard		XSTest		Avg.	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1
Ling-3.0-tiny								
GPT-OSS-120B	0.7996	0.6691	0.9039	0.4857	0.9200	0.4088	0.8745	0.5212
Qwen3Guard-Stream-8B	0.8068	0.6766	0.9226	0.5959	0.9193	0.2800	0.8829	0.5175
YuFeng-XGuard-Reason-8B	0.8195	0.7358	0.9141	0.6272	0.9021	0.3907	0.8786	0.5846
SingProbe	0.7375	0.6600	0.8506	0.4875	0.8386	0.2703	0.8089	0.4726
Ling-3.0-flash								
GPT-OSS-120B	0.9513	0.8112	0.9678	0.5630	0.9761	0.3846	0.9651	0.5863
Qwen3Guard-Stream-8B	0.9358	0.7315	0.9755	0.5782	0.9873	0.2609	0.9662	0.5235
YuFeng-XGuard-Reason-8B	0.9391	0.7925	0.9661	0.5885	0.9783	0.4082	0.9612	0.5964
SingProbe	0.9302	0.7586	0.9747	0.6364	0.9873	0.5405	0.9641	0.6452

distinctions emerge. We therefore use adaptive confidence weighting as the default training objective.

4.7 Generation-time Applications of SingProbe

4.7.1 Application 1: SingProbe for Online Content Detection

The preceding evaluations are conducted offline: each detector receives a fixed query–response pair under teacher forcing, allowing its predictions to be compared against pre-existing labels. In practical deployment, however, the base model generates responses autoregressively, and the guardrail must assess content produced by the model itself as generation unfolds. We therefore conduct an online evaluation in which Ling-3.0 freely generates responses while SingProbe produces detection signals alongside decoding. We study this setting for both response safety and hallucination detection.

Online Safety Detection. We compare SingProbe with GPT-OSS-120B (Agarwal et al., 2025) (as an LLM judge), Qwen3Guard-Stream-8B, and YuFeng-XGuard-Reason-8B on HarmBench, WildGuard, and XSTest. Because the safety labels of freely generated responses are not available a priori, we adopt a leave-one-out evaluation protocol. For each evaluated detector, the majority vote of the other three detectors is used as its pseudo-ground-truth label, ensuring that the evaluated detector does not contribute to its own reference label. We then report Accuracy and F1 against this leave-one-out consensus.

As shown in Table 8, SingProbe performance improves substantially with a stronger base model. With Ling-3.0-tiny, SingProbe obtains an average Accuracy of 0.8089 and an average F1 of 0.4726, which trails the stronger standalone baselines under this leave-one-out protocol. With Ling-3.0-flash, however, SingProbe delivers the strongest overall performance, reaching an average Accuracy of 0.9641 and an average F1 of 0.6452. It outperforms the strongest baseline average F1 (YuFeng-XGuard-Reason-8B at 0.5964) by 4.88 points and obtains the best F1 on both WildGuard (0.6364) and XSTest (0.5405). These results demonstrate that SingProbe remains effective when its predictions are produced during free autoregressive generation, rather than from fixed responses under teacher forcing.

Note that the comparatively modest F1 scores are largely attributable to the severe class imbalance in this online setting: most freely generated responses are Safe, leaving relatively few Unsafe examples. Under such a low positive-class prevalence, even a small number of false positives or disagreements among the detectors can substantially reduce precision and, consequently, the F1 score. The online F1 values should therefore be interpreted together with Accuracy and primarily used for relative comparison among methods evaluated under the same protocol, rather than directly compared with results on more balanced offline benchmarks.

Online Hallucination Detection. In the online hallucination evaluation, the model first performs autoregressive rollout on each test set to generate responses. An LLM, specifically Qwen3.5-397B-A17B in our experiments, then judges whether each response is correct based on the question, the model response, and the ground-truth answer, producing a binary label of whether hallucination occurs. Using these labels, we compute the hallucination-detection AUC for five methods: DRIFT (Bhatnagar et al., 2026), HaMI (Niu et al.,

Table 9: Online response hallucination detection under free autoregressive generation. Generated responses are assigned binary hallucination labels by Qwen3.5-397B-A17B based on the question, generated response, and reference answer. Values report response-level ROC AUC. The best result within each backbone is highlighted in bold.

Method	AA-Omniscience	BBH	GSM8K	MATH-500	NQ-Open	SQuAD	TruthfulQA	Avg.
Ling-3.0-tiny								
DRIFT	0.6380	0.6058	0.6446	0.4437	0.7482	0.6180	0.7229	0.6316
HaMI	0.5643	0.6431	0.5386	0.4185	0.6603	0.5570	0.5859	0.5668
SAPLMA-raw	0.5928	0.5924	0.6368	0.5448	0.5882	0.5616	0.5661	0.5832
SAPLMA-response	0.6695	0.6767	0.5426	0.4348	0.7089	0.6076	0.6411	0.6116
SingProbe	0.6032	0.5891	0.7328	0.7292	0.8136	0.6009	0.6813	0.6786
Ling-3.0-flash								
DRIFT	0.6662	0.5808	0.6193	0.6348	0.6852	0.7420	0.6536	0.6546
HaMI	0.5312	0.5802	0.6295	0.5519	0.6888	0.7143	0.6389	0.6193
SAPLMA-raw	0.6799	0.4872	0.6113	0.4877	0.6062	0.7339	0.5879	0.5992
SAPLMA-response	0.5188	0.5843	0.6495	0.6079	0.7125	0.7247	0.6699	0.6382
SingProbe	0.7353	0.4884	0.7702	0.7900	0.7792	0.8597	0.6668	0.7271

2025), SAPLMA-raw, SAPLMA-response (Azaria & Mitchell, 2023), and SingProbe, across seven datasets: AA-Omniscience (Jackson et al., 2025), BBH (Suzgun et al., 2023), GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021), NQ-Open (Lee et al., 2019), SQuAD (Rajpurkar et al., 2016), and TruthfulQA (Lin et al., 2022). Results are reported on two backbones, Ling-3.0-tiny and Ling-3.0-flash.

The results show that SingProbe achieves a clear and consistent advantage on both backbones, attaining the best average AUC in every case. On Ling-3.0-tiny, it reaches 0.6786, leading the second-best method DRIFT (0.6316) by an absolute margin of 0.047. On Ling-3.0-flash, it reaches 0.7271, leading DRIFT (0.6546) by 0.073. As backbone capability increases, SingProbe’s performance improves from 0.6786 to 0.7271, and its lead over the second-best method widens further, indicating that SingProbe can make fuller use of the internal signals from stronger backbone models.

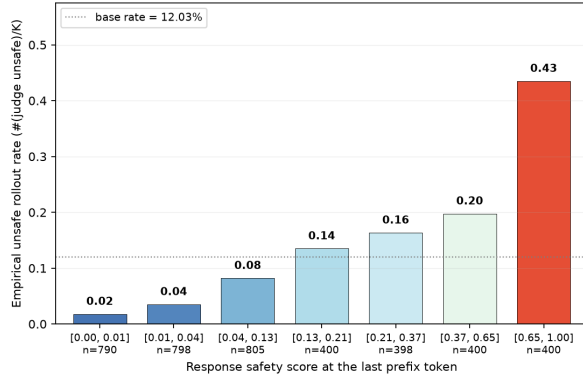
SingProbe’s advantage is also evident on individual datasets. On Ling-3.0-flash, SingProbe ranks first on five of the seven datasets: AA-Omniscience, GSM8K, MATH-500, NQ-Open, and SQuAD. Among them, the AUC on four datasets, AA-Omniscience, GSM8K, MATH-500, and SQuAD, exceeds 0.73, with SQuAD reaching 0.8597. On Ling-3.0-tiny, SingProbe also leads comprehensively on knowledge- or reasoning-intensive tasks such as GSM8K, MATH-500, and NQ-Open, achieving 0.7328, 0.7292, and 0.8136, respectively. In contrast, the baselines achieve local best results only on a few individual datasets and do not show a consistent advantage across backbones. Although SAPLMA-response performs relatively well on some BBH and TruthfulQA settings, its average AUC is 0.067 lower than SingProbe on tiny and 0.089 lower on flash; the gaps for HaMI and SAPLMA-raw are even larger.

4.7.2 Application 2: SingProbe for Constrained Safe Decoding

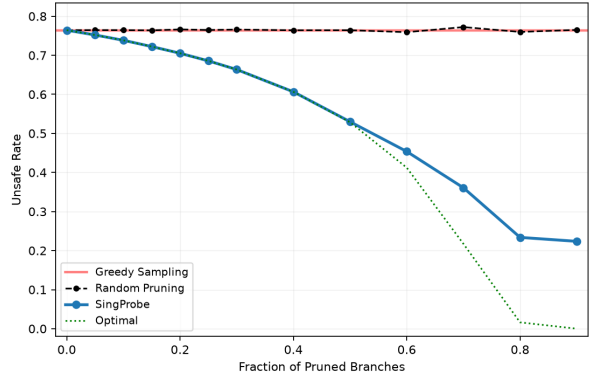
Beyond detecting unsafe content that has already appeared, an intrinsic streaming guardrail may also reveal how the safety of a response is likely to evolve. We study this possibility through two sequential research questions. First, *can SingProbe predict the safety tendency of future generations from the current prefix?* Second, if the score is predictive of future risk, *can it be used to guide safe decoding by terminating risky branches before they continue?* Figure 5 summarizes the corresponding rollout prediction and branch-pruning experiments.

Experimental Settings. We use Ling-3.0-flash as the frozen generation model and its SingProbe head as the safety probe. The response-unsafety score is streamed directly from the hidden states produced during decoding through the same SGLang serving pipeline, without an additional forward pass. For every evaluated prefix, we sample eight continuations. An independent YuFeng-XGuard-Reason-8B judge assigns a binary Safe or Unsafe label to each completed rollout. It is worth noting that the safety of the generated responses is assessed by an external judge and therefore may not constitute ground-truth labels. Nevertheless, the observed increase or decrease in the unsafe response rate still provides meaningful evidence of SingProbe’s contribution to safety.

Can SingProbe predict future generation risk? We operationalize this question by testing whether the score



(a) Predicting future generation risk from annotated safe prefixes. Starting from each query and its annotated safe response prefix, we group subsequent rollouts by the SingProbe score at the final safe-prefix token and report the empirical unsafe rates of their rollouts. Higher safe-prefix scores consistently correspond to greater unsafe risk in future continuations.



(b) SingProbe-guided branch pruning from annotated safe prefixes. Starting from each query and its annotated safe response prefix, we sample intermediate chunks to form candidate decoding branches. Branches with the highest chunk-end SingProbe scores are pruned before further continuation. The resulting policy substantially reduces the unsafe rate compared with greedy sampling and random pruning and closely approaches the optimal.

Figure 5: Anticipatory safety prediction and constrained safe decoding with SingProbe.

at the current *safe* prefix correlates with the fraction of future continuations that eventually become unsafe. Specifically, we collect 500 query–prefix pairs, including 350 unsafe-origin and 150 safe-origin examples. Each input consists of a query followed by an annotated safe response prefix, allowing us to test whether SingProbe can anticipate a future transition to unsafe content while the observed response remains safe. We record SingProbe’s response-unsafety score at the final token of this safe prefix, before any further continuation is sampled, and then generate eight independent rollouts. Their judged labels define the empirical future unsafe rate of the query–prefix pair, $\hat{p}_{\text{unsafe}} = n_{\text{unsafe}}/8$.

Figure 5a shows a clear monotonic relationship between the safe-prefix score and subsequent rollout safety. The empirical unsafe rate rises from approximately 0.02 in the lowest-score interval to 0.43 in the highest, which is roughly $3.6\times$ the overall base rate of 0.1203 and over $21\times$ the rate in the lowest-score group. The increase becomes particularly pronounced in the higher-score intervals, rising from 0.08 to 0.14, 0.16, 0.20, and finally 0.43. Because every observed prefix is annotated as safe and the score is recorded before continuation, this trend cannot be explained by SingProbe merely reacting to harmful content that has already appeared. Instead, it indicates that the hidden state at a safe prefix contains information about the conditional safety tendency of its possible continuations.

Can SingProbe guide safe decoding? Motivated by this predictive relationship, we next test whether the score can identify risky decoding branches early enough to prune them. As in the first experiment, each root consists of a query followed by an annotated safe response prefix. We select 16 conflict query–prefix roots for which three to five of the initial eight rollouts are unsafe, ensuring that every root admits both safe and unsafe continuations. Starting from each query and safe prefix, we sample 24 distinct 64-token chunks, yielding 384 intermediate branch prefixes. At the end of each chunk, we record SingProbe’s response-unsafety score before sampling any further tokens and then generate eight continuations, producing 3,072 judged rollouts. We simulate constrained decoding by removing the fraction p of branches with the highest SingProbe unsafety scores and measuring the unsafe rate among continuations from the retained branches. As shown in Figure 5b, this SingProbe-guided policy is compared with random pruning averaged over 200 permutations and an empirical-risk oracle that removes branches with the highest observed unsafe rates.

Guard-guided pruning consistently reduces the unsafe rate of the retained rollout pool, whereas random pruning leaves it nearly unchanged at approximately 0.76. When pruning 50% of the branches, SingProbe lowers the retained-pool unsafe rate to 0.53, a relative reduction of approximately 30% compared with random pruning, and essentially matches the oracle rate of 0.53. The unsafe rate continues to decrease as more high-scoring branches are removed, reaching approximately 0.22 when 90% of branches are pruned. The SingProbe and oracle curves nearly overlap up to a pruning fraction of about 0.5 and diverge only

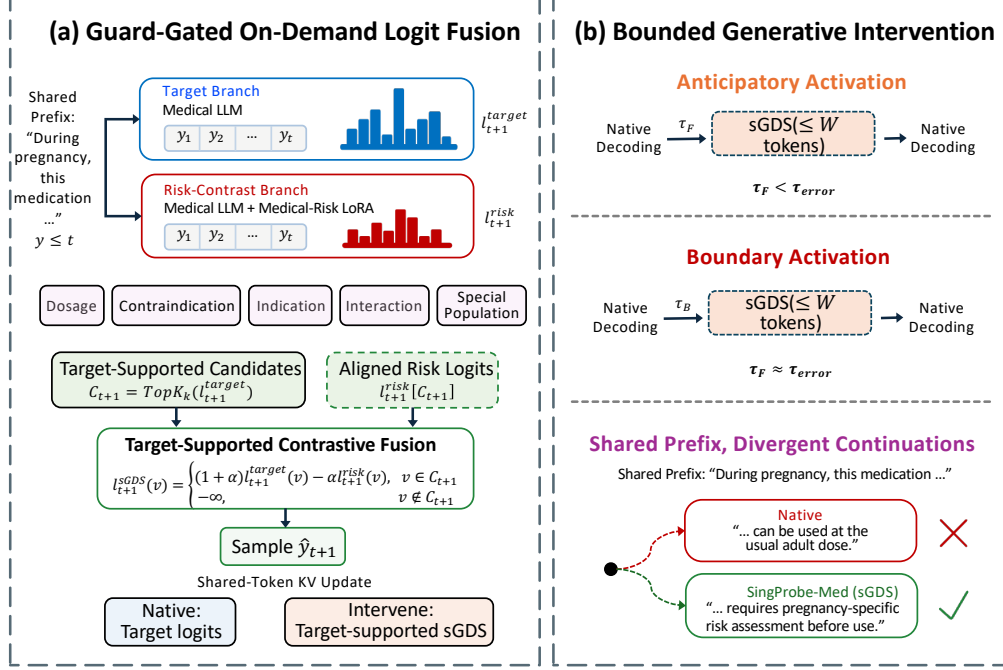


Figure 6: **SingProbe-Med intervention overview.** (a) Once admitted, target-supported sGDS contrasts target and medical risk-pattern logits while keeping both branches synchronized through a shared sampled token. (b) Future- or Boundary-based activation opens a bounded intervention window, after which generation returns to native decoding.

at aggressive pruning ratios, where the oracle drops toward 0 while SingProbe plateaus around 0.22–0.23. This suggests that SingProbe is highly accurate at identifying the highest-risk branches but is less precise in ranking the remaining lower-risk branches. Overall, these results demonstrate that SingProbe’s anticipatory signal is not only correlated with future safety outcomes but can also serve as a practical control signal for constrained safe decoding.

5 SingProbe-Med: On-Demand Medical-Risk Intervention

5.1 Motivation and Framework Overview

The preceding sections introduce SingProbe as a lightweight guardrail for a frozen LLM during autoregressive generation. We now study how such token-level internal-state signals can be used not only for detection, but also to control *when* a generation-time intervention should modify subsequent decoding.

Medical generation provides a representative setting in which selective intervention is particularly important. A harmful recommendation may emerge only after a clinically sound prefix, making query-level screening insufficient and post-hoc review too late to prevent the error from being generated (Draelos et al., 2026; Mishra et al., 2024b; Singhal et al., 2023; Yang et al., 2024). At the same time, applying a corrective decoding mechanism throughout the entire response is undesirable: always-on steering can perturb otherwise correct medical content and incurs persistent decoding cost. An effective intervention system should therefore answer two complementary questions: *when should intervention be activated, and how should generation be modified once intervention is needed?*

SingProbe-Med separates these two responsibilities. As illustrated in Figure 6, SingProbe-Med is responsible for deciding when to intervene by continuously monitoring the evolving generation trajectory through lightweight internal-state signals. Once the admission condition is satisfied, a target-supported intervention mechanism named support-constrained Guided Decoding Steering (sGDS) determines how to intervene by contrastively re-ranking the target model’s candidate tokens over a localized risky suffix. The intervention is bounded in duration, after which generation returns to the target model’s native decoding. This design separates inexpensive continuous monitoring from the more expensive corrective decoding mechanism, avoiding persistent dual-branch inference on generations for which no intervention is needed.

Table 10: Signal semantics in the SingProbe-Med intervention policy.

Signal	Scope	Intervention semantics
Intervention Eligibility E_t	Intervention applicability	Determines whether the current prefix expresses a clinically consequential proposition for which medical-risk intervention is applicable.
Global Risk G_t	Response trajectory	Estimates whether the evolving response is moving toward a harmful medical conclusion.
Future Risk F_t	Pre-error timing	Indicates that a confirmable medical error is likely within the next 1–32 tokens, allowing intervention before the error is committed.
Error Boundary B_t	Error-onset timing	Marks the first prefix at which a medical error becomes confirmable, allowing immediate containment when advance evidence is unavailable.

We instantiate this framework on AntAngelMed (AntAngelMed Team, 2025), a 100B-parameter MoE medical LLM. We next describe the two components separately: the SingProbe-Med admission mechanism for determining *when to intervene*, followed by target-supported sGDS for determining *how to intervene*.

5.2 SingProbe-Med: When to Intervene

At decoding step t , SingProbe-Med derives four complementary signals from the current prefix $y_{\leq t}$. Rather than serving as independent prediction tasks, these signals correspond to different decisions required by the intervention controller. The semantics of the four signals are summarized in Table 10.

- **Intervention Eligibility** E_t determines whether the current prefix contains a clinically consequential proposition for which medical-risk intervention is applicable.
- **Global Risk** G_t estimates whether the evolving response trajectory is moving toward a harmful medical conclusion. The remaining two signals provide more localized timing information.
- **Future Risk** F_t indicates whether a confirmable medical error is likely to emerge within the next 1–32 tokens, enabling intervention before the error is committed.
- **Error Boundary** B_t identifies the first prefix at which a medical error becomes confirmable, allowing immediate containment when sufficiently early predictive evidence is unavailable.

These signals are combined through a fixed evidential hierarchy to form an admission policy. Intervention of the next uncommitted token is activated according to

$$a_{t+1} = \mathbb{I}[E_t \geq \gamma_E \wedge (G_t \geq \gamma_G \vee (G_t \geq \gamma_C \wedge (F_t \geq \gamma_F \vee B_t \geq \gamma_B)))] . \quad (8)$$

This rule follows an $E \rightarrow G \rightarrow (F/B)$ hierarchy motivated by the asymmetric cost of false intervention. E_t first restricts intervention to prefixes for which risk-directed steering is applicable. G_t then provides trajectory-level evidence of emerging medical harm. When the global-risk signal is sufficiently strong, intervention can be admitted directly to avoid unnecessary delay. Under intermediate global evidence, the shorter-horizon F_t and B_t signals refine the intervention timing.

Importantly, F_t and B_t are complementary entry signals rather than sequential stages. F_t supports *anticipatory intervention* before a medical error becomes explicit, whereas B_t provides *boundary-level containment* at the earliest prefix where the error can be confirmed. All four signals are computed in parallel; the hierarchy therefore specifies evidential precedence rather than a sequential execution pipeline. The operating thresholds and release conditions are fixed on the development split before final evaluation.

5.3 Target-Supported sGDS: How to Intervene

Once SingProbe-Med determines that intervention is warranted, we steer the subsequent generation by explicitly modeling and suppressing medical-risk preferences in the decoding distribution. Specifically, we construct a risk-pattern branch by adapting the target model with a medical-risk LoRA adapter (Hu et al., 2022), and use the difference between the target and risk-pattern logits to guide contrastive intervention. In the following sections, we first define the medical-risk patterns covered by the intervention. We then describe how the corresponding risk-pattern branch is constructed and how its logits are incorporated into target-supported sGDS.

Table 11: Medical-risk categories covered by the intervention branch.

Category	Intervention target
Dosage and administration	Incorrect dose, frequency, duration, or usage instruction.
Special-population contraindication	A recommendation that conflicts with constraints associated with a clinically relevant patient population.
Wrong indication	A medication or treatment recommendation that does not match the stated condition.
Dermatology wrong indication	An indication mismatch involving a dermatologic condition or therapy.
Dermatology special-population contraindication	A dermatologic recommendation that conflicts with population-specific safety constraints.

Medical-risk Intervention Scope. We define the intervention scope using five operational medical-risk categories: dosage and administration errors, special-population contraindications, wrong indications, dermatology-specific wrong indications, and dermatology-specific special-population contraindications. These categories are used to organize the risk-pattern adaptation data and jointly construct the training set for the medical intervention branch. The sampler maintains approximately balanced coverage across the five categories. Table 11 summarizes the corresponding intervention targets. The resulting risk-pattern dataset is used to adapt the medical intervention branch described next.

Medical Risk-pattern Branch. We construct the medical risk-pattern branch by adapting the deployed AntAngelMed model with a LoRA module on the risk-pattern dataset described above. The adaptation follows a standard supervised fine-tuning style objective: given a medical prompt x and a risk-pattern response $y = (y_1, \dots, y_T)$, we optimize only the LoRA parameters Δ_{risk} while keeping the base-model parameters θ frozen. Specifically, the branch is trained with the autoregressive language-modeling loss

$$\mathcal{L}_{\text{risk}} = - \sum_{t=1}^T \log p_{\theta + \Delta_{\text{risk}}} (y_t \mid x, y_{<t}), \quad (9)$$

where the training responses are drawn from the five medical-risk categories defined above. Optimizing this objective increases the adapted branch’s likelihood on continuations that exhibit the corresponding medical-risk patterns, thereby encoding these patterns into the learned LoRA parameters.

After adaptation, the original model and the risk-pattern-adapted model form the two branches used during intervention. Let f_{θ} denote the original AntAngelMed model and Δ_{risk} the learned medical-risk LoRA parameters. Given the realized decoding context $c_t = (x, y_{\leq t})$, they produce next-token logits as

$$\ell_{t+1}^{\text{tar}} = f_{\theta}(c_t), \quad \ell_{t+1}^{\text{risk}} = f_{\theta + \Delta_{\text{risk}}}(c_t). \quad (10)$$

Because the adapted branch is explicitly trained on risk-pattern continuations, it assigns relatively greater support to tokens and continuations that are compatible with the covered medical-risk patterns. We therefore use it as an *anti-expert*: ℓ_{t+1}^{risk} provides a token-level risk preference that can be contrasted with the target-model logits during intervention, rather than being interpreted as an explicit medical-error probability or an independently generated correction.

On-demand Target-supported sGDS. With the medical risk-pattern branch defined above, we use its next-token logits as an anti-expert signal to modify the target model only when intervention is activated. At each activated decoding step, the target and risk-pattern branches evaluate the same realized context $c_t = (x, y_{\leq t})$, producing ℓ_{t+1}^{tar} and ℓ_{t+1}^{risk} for the same uncommitted token. We then contrast these two distributions while restricting the intervention to candidates already supported by the target model. Specifically, the target branch first defines the admissible candidate set

$$C_{t+1} = \text{TopK}(\ell_{t+1}^{\text{tar}}, k). \quad (11)$$

Within this support, sGDS computes

$$\ell_{t+1}^{\text{sGDS}}(v) = \begin{cases} (1 + \alpha)\ell_{t+1}^{\text{tar}}(v) - \alpha\ell_{t+1}^{\text{risk}}(v), & v \in C_{t+1}, \\ -\infty, & v \notin C_{t+1}. \end{cases} \quad (12)$$

The resulting ℓ_{t+1}^{sGDS} serves as the final next-token logits, from which the sampling distribution is obtained and the next token is selected. Equivalently, sGDS adds the contrastive direction $\alpha(\ell_{t+1}^{\text{tar}} - \ell_{t+1}^{\text{risk}})$ to the native target

logits within C_{t+1} . Since the risk-pattern branch is adapted to favor continuations exhibiting the covered medical-risk patterns, candidates that receive relatively stronger support from this branch are suppressed, while candidates favored more strongly by the target branch are promoted relative to their competitors. The top- k restriction keeps the target model as the source of admissible next tokens, so the risk branch only re-ranks plausible target-supported candidates rather than introducing its own off-support continuations.

The two branches operate as paired conditional scorers rather than independently decoded models. They begin from the same prompt and realized response prefix while maintaining separate model states and KV caches. At each activated step, a single token $\hat{y}_{t+1}$ is sampled from the fused sGDS distribution and appended to both branches. This shared-token update keeps their prefixes synchronized, ensuring that subsequent target and risk-pattern logits continue to describe the same generation history. If the branches were decoded independently, their prefixes would diverge and the resulting logit difference would no longer isolate the learned risk preference.

When $a_{t+1} = 0$, generation proceeds directly from the unmodified target-model logits and the risk-pattern branch does not participate in logit fusion. Once intervention is activated, it opens an intervention episode

$$e = (\tau, W, k, \alpha), \quad (13)$$

where τ denotes the activation position, W bounds the intervention duration, k determines the target-supported candidate set, and α controls the strength of the contrastive correction. We evaluate both a full intervention and a bounded variant. The full intervention uses $k = 50$ and $\alpha = 2$ until release, whereas the bounded variant uses $k = 100$, $\alpha = 3$, and $W = 64$. After the episode is released, decoding returns to target-only logits on the already modified prefix.

Thus, the intervention is on demand at two levels. SingProbe-Med determines *when* the more expensive dual-branch decoding should be activated, while target-supported sGDS determines *how* the next-token distribution should be modified once activated. By confining both the activation interval and the admissible vocabulary, the resulting intervention suppresses learned medical-risk preferences only over the risk-relevant portion of generation.

5.4 Evaluation

We evaluate SingProbe-Med along four complementary quantitative axes: admission selectivity and timing, medical error correction, general capability preservation, and inference efficiency. We further provide a qualitative case study to illustrate how the resulting intervention modifies representative medical-risk continuations. All admission thresholds and episode-release conditions are selected on the development split and frozen before evaluation on the held-out medical set. No threshold is reselected on the test set or separately for an individual capability benchmark.

5.4.1 Admission Selectivity and Timing

We first evaluate whether the admission mechanism can identify trajectories that warrant intervention and trigger sufficiently early. “Harm” denotes a trajectory containing a target medical risk, whereas “White” denotes a trajectory for which intervention should not be triggered. Let A_i indicate whether trajectory i opens at least one intervention episode. At a fixed operating point, we report

$$\begin{cases} \text{White FPR} = \Pr(A_i = 1 \mid i \in \text{White}), \\ \text{Harm recall} = \Pr(A_i = 1 \mid i \in \text{Harm}), \\ \text{Precision} = \Pr(i \in \text{Harm} \mid A_i = 1). \end{cases} \quad (14)$$

We additionally report ROC-AUC, Average Precision (AP), and PR-AUC from the complete trajectory-level ranking on the held-out set. For the White-FPR operating points $q \in \{10\%, 5\%, 1\%\}$, thresholds are selected on the development set and frozen before evaluating Harm recall and precision on the held-out set.

For the combined detector, Intervention Eligibility acts as a binary gate on the Global Risk score. Specifically, we define the gated risk score as

$$S_{E+G} = \mathbb{I}(E > \gamma_E) \sigma(G), \quad (15)$$

where γ_E is the Intervention Eligibility threshold and $\sigma(\cdot)$ denotes the sigmoid function. Thus, Global Risk contributes to the ranking only when the corresponding prefix is considered eligible for medical-risk intervention.

Table 12 compares Global Risk alone with this eligibility-gated Global Risk score. Applying the Intervention Eligibility gate improves ROC-AUC, AP, and PR-AUC by 0.025, 0.012, and 0.011, respectively, indicating

Table 12: Operating characteristics of the SingProbe-Med admission signals on the held-out medical set.

(a) Ranking quality				(b) Constrained operating points (%)		
Detector	ROC-AUC	AP	PR-AUC	White FPR	Harm recall	Precision
Global Risk	0.828	0.903	0.903	$\leq 10\%$	51.70	92.34
Intervention Eligibility + Global Risk	0.853	0.915	0.914	$\leq 5\%$	32.44	93.93
				$\leq 1\%$	11.98	96.77

Table 13: Paired medical correction results (%). “Activated” conditions on trajectories where SingProbe-Med opens an sGDS episode.

Metric	Full intervention		64-token window	
	Overall	Activated	Overall	Activated
Broad mitigation (ERROR $\rightarrow$ non-ERROR)	25.03	30.27	24.30	29.39
Strict repair (ERROR $\rightarrow$ SAFE)	12.58	15.22	11.72	14.17
Safe regression	1.84	6.78	2.53	9.32

that suppressing risk scores on intervention-ineligible prefixes improves trajectory selection. Tightening the allowed White FPR from 10% to 1% increases precision from 92.34% to 96.77%, while Harm recall decreases from 51.70% to 11.98%, exposing a direct trade-off between intervention coverage and false-trigger control.

We further evaluate activation timing on Harm trajectories with a confirmable medical error boundary. Let T^{confirm} denote the first prefix at which the medical error becomes confirmable. Among Harm trajectories, 54.99% of first activations occur at or before T^{confirm} , meaning that the policy opens a correction window no later than the confirmable error boundary for more than half of the risky trajectories. Evaluating all four SingProbe-Med signals over 3,044,056 token states takes 0.019749 ms/token, corresponding to approximately 50,635 token states per second.

5.4.2 Medical Error Correction

We next evaluate whether an admitted intervention corrects the final medical outcome. Each test case is matched by prompt between target-only decoding and an intervention configuration. Let z_i^0 and z_i^m denote the three-way adjudicated outcomes of the baseline and intervened responses in {ERROR, SAFE, UNCERTAIN}. We define *broad mitigation* as a baseline ERROR changed to either SAFE or UNCERTAIN, *strict repair* as ERROR $\rightarrow$ SAFE, and *safe regression* as a baseline SAFE changed to a non-SAFE outcome. “Overall” reports these transitions over the corresponding baseline population in the complete test set, whereas “Activated” additionally conditions on trajectories where at least one sGDS episode is opened. We compare two intervention configurations. The full intervention uses $k = 50$ and $\alpha = 2$ until release, whereas the bounded configuration uses $k = 100$, $\alpha = 3$, and a maximum intervention window of $W = 64$ tokens.

Table 13 reports the paired correction results. The full intervention mitigates 25.03% of baseline erroneous generations over the complete test set, including 12.58% that are strictly repaired from ERROR to SAFE. On trajectories where intervention is actually activated, broad mitigation and strict repair increase to 30.27% and 15.22%, respectively, indicating that the admission mechanism concentrates intervention on trajectories with greater correction potential. Restricting intervention to a 64-token window preserves most of this benefit. The bounded configuration achieves 24.30% broad mitigation and 11.72% strict repair overall, retaining 97.1% and 93.2% of the corresponding full-intervention results. This suggests that most useful corrections occur in the early suffix following activation rather than requiring persistent dual-branch decoding.

Intervention nevertheless introduces a non-zero risk of modifying originally safe generations. Reported safe regression is 1.84% overall for the full intervention and 2.53% for the 64-token configuration, with higher rates when conditioning specifically on activated trajectories. This trade-off further motivates selective admission rather than applying sGDS indiscriminately.

5.4.3 General Capability Preservation

We next examine whether selective intervention perturbs model capability on inputs for which medical-risk steering is unnecessary. We compare native AntAngelMed-100B, always-on sGDS, and SingProbe-Med across AIME24/25 (Maxwell-Jia, 2024; Zhang & Math-AI Team, 2025), HealthBench (Arora et al., 2025), CFBench (Zhang et al., 2025), Arena-Hard-v2 (Li et al., 2025a), AlignBench (Liu et al., 2024), and

Table 14: Capability scores and sGDS activation rates (in parentheses).

Method	AIME24/25 avg.	HealthBench	CFBench	Arena-Hard-v2	AlignBench	WritingBench
AntAngelMed-100B	66.67	88.80	84.31/59.00/69.50	52.06	8.435	8.351
Always-on sGDS	60.00 (100%)	88.41 (100%)	81.73/52.00/64.00 (100%)	46.19 (100%)	8.465 (100%)	8.138 (100%)
SingProbe-Med	66.67 (0%)	90.10 (4.69%)	84.31/59.00/69.50 (0%)	52.06 (0%)	8.435 (0%)	8.351 (0%)

Table 15: Request-level mean latency across inference configurations. Entries marked with $\sim$ are component-based estimates rather than directly measured end-to-end means.

Configuration	Mean (s)
AntAngelMed-100B, single branch	37.319
AntAngelMed-100B + SingProbe signals	$\sim$ 37.379
Full resident dual branch	60.934
64-token on-demand window	$\sim$ 39.162

WritingBench (Wu et al., 2026). Evaluation follows each benchmark’s native scoring convention: AIME24/25 averages the 2024 and 2025 scores, CFBench reports CSR/ISR/PSR, and the remaining benchmarks retain their native scales. For intervention-based methods, the activation rate is defined as the percentage of benchmark examples that open at least one sGDS episode.

As shown in Table 14, SingProbe-Med activates on only 4.69% of HealthBench examples and on none of the reported examples from AIME24/25, CFBench, Arena-Hard-v2, AlignBench, or WritingBench. Its capability scores remain broadly comparable to native AntAngelMed-100B across these evaluations. In contrast, always-on sGDS modifies every example and records lower scores than SingProbe-Med on all six benchmarks, with particularly noticeable degradation on AIME24/25 and Arena-Hard-v2.

5.4.4 Inference Efficiency

Finally, we evaluate whether on-demand intervention reduces the inference cost of dual-branch decoding. We compare four AntAngelMed-100B inference configurations: native single-branch decoding, single-branch decoding with continuous SingProbe-Med monitoring, a fully resident dual-branch configuration, and a bounded 64-token intervention window. The single-branch and resident dual-branch entries are reference measurements. The target-plus-SingProbe-Med and 64-token entries are component-based estimates derived from the measured single-branch latency and the corresponding additional signal or active-window cost, and are therefore marked with $\sim$ in Table 15.

Continuous SingProbe-Med scoring contributes an estimated 0.060 seconds beyond the 37.319-second single-branch reference, yielding approximately 37.379 seconds before intervention cost is considered. Maintaining the full dual branch throughout generation increases the request-level mean latency to 60.934 seconds, an additional 23.615 seconds over native decoding. In comparison, a 64-token intervention window introduces an estimated 1.843-second active-window cost, giving approximately 39.162 seconds when added to the single-branch reference. These results show that bounding sGDS to the admitted suffix converts persistent dual-branch computation into a limited intervention cost. Together with the correction results above, they demonstrate that most of the safety benefit of full intervention can be retained without continuously executing the second branch throughout generation.

6 Conclusion

In this work, we introduced SingProbe, an intrinsic guardrail that directly leverages hidden states produced by the base LLM to jointly monitor query intent, response safety, and hallucination risk throughout generation. By reusing representations already computed during decoding, SingProbe avoids redundant text encoding and enables fine-grained streaming detection with substantially lower deployment complexity than standalone guardrail models. We also introduced SingStreamBench to evaluate the temporal behavior of streaming safety detectors, particularly their ability to remain silent on benign prefixes and identify the onset of unsafe content. Across extensive offline and online evaluations, SingProbe achieves strong performance on safety and hallucination detection while maintaining very low false-positive rates.

More importantly, our results suggest that the value of intrinsic guardrails extends beyond passive classification. SingProbe scores contain information about the safety tendency of future continuations and can therefore serve as control signals for constrained decoding. SingProbe-Med further demonstrates how such

internal-state signals can support selective, on-demand intervention in a high-stakes domain, modifying generation only when and where intervention is warranted. Overall, SingProbe provides a lightweight interface between the internal representations of an LLM and generation-time safety mechanisms, offering a path toward guardrails that jointly improve monitoring granularity, computational efficiency, and actionable runtime control.

Authors

Shiwen Cui, Jianjie Jiang, Guoyi Li, Jinzhen Lin, Changhua Meng, Shuo Shao, Jiashui Wang, Weiqiang Wang, Weixi Wu, Zhuoer Xu, Zihan Yan, Shenglin Yin, Xinlei Ying, Zhe Zhao, Jing Zhou.

(All authors are listed alphabetically by last name.)

Author Contributions

Shuo Shao, Zhuoer Xu, and Zihan Yan contributed to the safety aspect, and Jianjie Jiang, Weixi Wu, Shenglin Yin, Xinlei Ying, and Zhe Zhao contributed to the hallucination aspect of SingProbe. Jinzhen Lin developed the supporting infrastructure. Guoyi Li and Jing Zhou developed SingProbe-med. Shiwen Cui, Changhua Meng, Jiashui Wang, and Weiqiang Wang supervised this project.

References

- Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- AntAngelMed Team. AntAngelMed: A high-performance medical language model with efficient MoE-powered clinical reasoning, 2025. URL <https://huggingface.co/MedAIBase/AntAngelMed>.
- Rahul K Arora, Jason Wei, Rebecca Soskin Hicks, Preston Bowman, Joaquin Quiñero-Candela, Foivos Tsimpourlas, Michael Sharman, Meghan Shah, Andrea Vallone, Alex Beutel, et al. Healthbench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*, 2025.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Amos Azaria and Tom Mitchell. The internal state of an llm knows when it’s lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 967–976, 2023.
- Rohan Bhatnagar, Youran Sun, Chi Andrew Zhang, Yixin Wen, and Haizhao Yang. Drift: Detecting representational inconsistencies for factual truthfulness. *arXiv preprint arXiv:2601.14210*, 2026.
- Xiang Chen, Duanzheng Song, Honghao Gui, Chenxi Wang, Ningyu Zhang, Yong Jiang, Fei Huang, Chengfei Lv, Dan Zhang, and Huajun Chen. Factchd: Benchmarking fact-conflicting hallucination detection. *arXiv preprint arXiv:2310.12086*, 2023.
- Minseok Choi, Dongjin Kim, Seungbin Yang, Subin Kim, Youngjun Kwak, Juyoung Oh, Jaegul Choo, and Jungmin Son. Expguard: LLM content moderation in specialized domains. In *International Conference on Learning Representations*, 2026.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. Free dolly: Introducing the world’s first truly open instruction-tuned llm, 2023. URL <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm>.
- Hoagy Cunningham, Jerry Wei, Zihan Wang, Andrew Persic, Alwin Peng, Jordan Abderrachid, Raj Agarwal, Bobby Chen, Austin Cohen, Andy Dau, et al. Constitutional classifiers++: Efficient production-grade defenses against universal jailbreaks. *arXiv preprint arXiv:2601.04603*, 2026.
- Peng Ding, Jun Kuang, Zongyu Wang, Xuezhi Cao, Xunliang Cai, Jiajun Chen, and Shujian Huang. Why not act on what you know? unleashing safety potential of llms via self-aware guard enhancement. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 6279–6299, 2025.
- Rachel L. Draelos, Samina Afreen, Barbara Blasko, Tiffany L. Brazile, Natasha Chase, Dimple Patel Desai, Jessica Evert, Heather L. Gardner, Lauren Herrmann, Aswathy Vaikom House, Stephanie Kass, Marianne Kavan, Kirshma Khemani, Amanda Koire, Lauren M. McDonald, Zahraa Rabeeah, and Amy Shah. Large language models provide unsafe answers to patient-posed medical questions. *npj Digital Medicine*, 9(1):241, 2026. doi: 10.1038/s41746-026-02428-5. URL <https://www.nature.com/articles/s41746-026-02428-5>.
- Nouha Dziri, Ehsan Kamalloo, Sivan Milton, Osmar R Zaiane, Mo Yu, Edoardo M Ponti, and Siva Reddy. Faithdial: A faithful benchmark for information-seeking dialogue. *Transactions of the Association for Computational Linguistics*, 10:1473–1490, 2022.
- Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. Vibe coding in practice: Motivations, challenges, and a future outlook – a grey literature review. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*, pp. 212–223, New York, NY, USA, 2026. Association for Computing Machinery. ISBN 9798400724268. doi: 10.1145/3786583.3786866. URL <https://doi.org/10.1145/3786583.3786866>.
- Shaona Ghosh, Prasoon Varshney, Makesh Narsimhan Sreedhar, Aishwarya Padmakumar, Traian Rebedea, Jibin Rajan Varghese, and Christopher Parisien. AEGIS2.0: A diverse AI safety dataset and risks taxonomy

- for alignment of LLM guardrails. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5992–6026, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.306. URL <https://aclanthology.org/2025.naacl-long.306/>.
- Google. Gemini 3. <https://blog.google/products-and-platforms/products/gemini/gemini-3/>, 2025.
- Ant Group. Singguard: A policy-adaptive multimodal llm guardrail with dynamic reasoning. *arXiv preprint arXiv:2606.22873*, 2026.
- Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. *Advances in neural information processing systems*, 37:8093–8131, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- Declan Jackson, William Keating, George Cameron, and Micah Hill-Smith. Aa-omniscience: Evaluating cross-domain knowledge reliability in large language models. *arXiv preprint arXiv:2511.13029*, 2025.
- Jiaming Ji, Mickel Liu, Josef Dai, Xuehai Pan, Chi Zhang, Ce Bian, Boyuan Chen, Ruiyang Sun, Yizhou Wang, and Yaodong Yang. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. *Advances in Neural Information Processing Systems*, 36:24678–24704, 2023.
- Jiaming Ji, Donghai Hong, Borong Zhang, Boyuan Chen, Josef Dai, Boren Zheng, Tianyi Qiu, Boxun Li, and Yaodong Yang. Pku-saferllm: Towards multi-level safety alignment for llms with human preference. *arXiv preprint arXiv:2406.15513*, 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.
- Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. Latent retrieval for weakly supervised open domain question answering. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 6086–6096, 2019.
- Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 34209–34231. PMLR, 2025a.
- Yang Li, Qiang Sheng, Yehan Yang, Xueyao Zhang, and Juan Cao. From judgment to interference: Early stopping llm harmful outputs via streaming content monitoring. *Advances in Neural Information Processing Systems*, 38:54305–54333, 2025b.
- Junyu Lin, Meizhen Liu, Xiufeng Huang, Jinfeng Li, Haiwen Hong, Xiaohan Yuan, Yuefeng Chen, Longtao Huang, Hui Xue, Ranjie Duan, et al. Yufeng-xguard: A reasoning-centric, interpretable, and flexible guardrail model for large language models. *arXiv preprint arXiv:2601.15588*, 2026.
- Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)*, pp. 3214–3252, 2022.

- Xiao Liu, Xuanyu Lei, Shengyuan Wang, Yue Huang, Andrew Feng, Bosi Wen, Jiale Cheng, Pei Ke, Yifan Xu, Weng Lam Tam, et al. Alignbench: Benchmarking chinese alignment of large language models. In *Annual Meeting of the Association for Computational Linguistics*, pp. 11621–11640, 2024.
- Junliang Luo, Tianyu Li, Di Wu, Michael Jenkin, Steve Liu, and Gregory Dudek. Hallucination detection and hallucination mitigation: An investigation. *arXiv preprint arXiv:2401.08358*, 2024.
- Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.
- Xingjun Ma, Yifeng Gao, Yixu Wang, Ruofan Wang, Xin Wang, Ye Sun, Yifan Ding, Hengyuan Xu, Yunhao Chen, Yunhan Zhao, et al. Safety at scale: A comprehensive survey of large model and agent safety. *Foundations and Trends in Privacy and Security*, 8(3-4):1–240, 2026.
- Potsawee Manakul, Adian Liusie, and Mark Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 9004–9017, 2023.
- Todor Markov, Chong Zhang, Sandhini Agarwal, Florentine Eloundou Nekoul, Theodore Lee, Steven Adler, Angela Jiang, and Lilian Weng. A holistic approach to undesired content detection in the real world. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pp. 15009–15018, 2023.
- Maxwell-Jia. Aime 2024 dataset. Hugging Face Dataset, 2024. URL https://huggingface.co/datasets/Maxwell-Jia/AIME_2024.
- Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, et al. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. In *International Conference on Machine Learning*, 2024.
- Timothee Mickus, Elaine Zosa, Raúl Vázquez, Teemu Vahtola, Jörg Tiedemann, Vincent Segonne, Alessandro Raganato, and Marianna Apidianaki. Semeval-2024 task 6: Shroom, a shared-task on hallucinations and related observable overgeneration mistakes. In *Proceedings of the 18th International Workshop on Semantic Evaluation*, pp. 1979–1993, 2024.
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*, 2018.
- Abhika Mishra, Akari Asai, Vidhisha Balachandran, Yizhong Wang, Graham Neubig, Yulia Tsvetkov, and Hannaneh Hajishirzi. Fine-grained hallucination detection and editing for language models. *arXiv preprint arXiv:2401.06855*, 2024a.
- Abhika Mishra, Akari Asai, Vidhisha Balachandran, Yizhong Wang, Graham Neubig, Yulia Tsvetkov, and Hannaneh Hajishirzi. Fine-grained hallucination detection and editing for language models. *arXiv preprint arXiv:2401.06855*, 2024b.
- Moonshot AI. Kimi k2.6. <https://huggingface.co/moonshotai/Kimi-K2.6>, 2026. Hugging Face model repository.
- Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, Kashun Shum, Randy Zhong, Juntong Song, and Tong Zhang. Ragtruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 10862–10878, 2024.
- Mengjia Niu, Hamed Haddadi, and Guansong Pang. Robust hallucination detection in llms via adaptive token selection. *Advances in Neural Information Processing Systems*, 38:126355–126377, 2025.
- OpenAI. Gpt-5. <https://openai.com/index/gpt-5-1/>, 2025.
- Weiwei Qi, Zefeng Wu, Zhilin Guo, Tianhang Zheng, Chaochao Lu, Liang He, Zhan Qin, and Kui Ren. Darwin: Evolving jailbreak adversary and guardrail for llm safety evaluation and protection. *arXiv preprint arXiv:2607.19829*, 2026.
- Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.

- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pp. 2383–2392, 2016.
- IBM Research. Granite guardian. *Technical Report*, 2024.
- Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. *arXiv preprint arXiv:2308.01263*, 2023.
- Mrinank Sharma, Meg Tong, Jesse Mu, Jerry Wei, Jorrit Kruthoff, Scott Goodfriend, Euan Ong, Alwin Peng, Raj Agarwal, Cem Anil, et al. Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. *arXiv preprint arXiv:2501.18837*, 2025.
- Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. Large language models encode clinical knowledge. *Nature*, 620(7972):172–180, 2023. doi: 10.1038/s41586-023-06291-2.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed H Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 13003–13051, 2023.
- Jialin Wu, Jiangyi Deng, Shengyuan Pang, Yanjiao Chen, Jiayang Xu, Xinfeng Li, and Wenyuan Xu. Legilimens: Practical and unified content moderation for large language model services. In *ACM SIGSAC Conference on Computer and Communications Security*, pp. 1151–1165, 2024.
- Yuning Wu, Jiahao Mei, Ming Yan, Chenliang Li, Shaopeng Lai, Yuran Ren, Zijia Wang, Ji Zhang, Mengyue Wu, Qin Jin, et al. Writingbench: A comprehensive benchmark for generative writing. *Advances in Neural Information Processing Systems*, 38, 2026.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yifan Yang, Qiao Jin, Robert Leaman, Xiaoyu Liu, Guangzhi Xiong, Maame Sarfo-Gyamfi, Changlin Gong, Santiago Ferrière-Steinert, W. John Wilbur, Xiaojun Li, Jiabin Yuan, Bang An, Kelvin S. Castro, et al. Ensuring safety and trust: Analyzing the risks of large language models in medicine. *arXiv preprint arXiv:2411.14487*, 2024. doi: 10.48550/arXiv.2411.14487.
- Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.
- Wenjun Zeng et al. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint arXiv:2407.21772*, 2024.
- Tao Zhang, Chenglin Zhu, Yanjun Shen, Wenjing Luo, Yan Zhang, Hao Liang, Fan Yang, Mingan Lin, Yujing Qiao, Weipeng Chen, et al. Cfbench: A comprehensive constraints-following benchmark for llms. In *Annual Meeting of the Association for Computational Linguistics*, pp. 32926–32944, 2025.
- Yifan Zhang and Math-AI Team. American invitational mathematics examination (aime) 2025, 2025. URL <https://huggingface.co/datasets/math-ai/aime25>.
- Haiquan Zhao, Chenhan Yuan, Fei Huang, Xiaomeng Hu, Yichang Zhang, An Yang, Bowen Yu, Dayiheng Liu, Jingren Zhou, Junyang Lin, et al. Qwen3guard technical report. *arXiv preprint arXiv:2510.14276*, 2025a.
- Jiachen Zhao, Jing Huang, Zhengxuan Wu, David Bau, and Weiyang Shi. Llm encode harmfulness and refusal separately. *Advances in Neural Information Processing Systems*, 38:140283–140318, 2025b.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37*, 2024.